C++ Chapter 2 — Functions and Files

Chapter 02 — C++ Basics: Functions and Files

02.01 — Introduction to functions

In the last chapter, we defined a function as a collection of statements that execute sequentially. While that is certainly true, that definition doesn't provide much insight into why functions are useful. Let's update our definition: A **function** is a reusable sequence of statements designed to do a particular job.

You already know that every executable program must have a function named `main()` (which is where the program starts execution when it is run). However, as programs start to get longer and longer, putting all the code inside the `main()` function becomes increasingly hard to manage. Functions provide a way for us to split our programs into small, modular chunks that are easier to organize, test, and use. Most programs use many functions. The C++ standard library comes with plenty of already-written functions for you to use – however, it's just as common to write your own. Functions that you write yourself are called **user-defined functions**.

Consider a case that might occur in real life: you're reading a book, when you remember you need to make a phone call. You put a bookmark in your book, make the phone call, and when you are done with the phone call, you return to the place you bookmarked and continue your book precisely where you left off.

C++ programs can work the same way (and borrow some of the same nomenclature). A program will be executing statements sequentially inside one function when it encounters a function call. A **function call** tells the CPU to interrupt the current function and execute another function. The CPU essentially "puts a bookmark" at the current point of execution, executes the function named in the function call, and then **returns** to the point it bookmarked and resumes execution.

> **Nomenclature**
>
> The function initiating the function call is the **caller**, and the function being **called** (executed) is the **callee**. A function call is also sometimes called an **invocation**, with the caller **invoking** the callee.

An example of a user-defined function

First, let's start with the most basic syntax to define a user-defined function. For the next few lessons, all user-defined functions will take the following form:

```
returnType functionName() // This is the function header (tells the compiler about the existence of the function)
{
    // This is the function body (tells the compiler what the function does)
}
```

The first line is informally called the **function header**, and it tells the compiler about the existence of a function, the function's name, and some other information that we'll cover in future lessons (like the return type).

- In this lesson, we'll use a *returnType* of `int` (for function `main()`) or `void` (otherwise). For now, don't worry about these, as we'll talk more about return types and return values in the next lesson (2.2 – Function return values (value-returning functions)).
- Just like variables have names, so do user-defined functions. The *functionName* is the name (identifier) of your user-defined function.
- The parentheses after the identifier tell the compiler that we're defining a function.

The curly braces and statements in-between are called the **function body**. This is where the statements that determine what your function does will go.

To call a function, we use the function's name followed by a set of parentheses (e.g. `functionName()` calls the function whose name is `functionName`). Conventionally, the parenthesis are placed adjacent to the function name (with no whitespace between them).

For now, a function must be defined before it can be called. We'll discuss ways to work around this in lesson 2.7 – Forward declarations and definitions.

Here is a sample program that illustrates a user-defined function being defined and called:

```
#include <iostream> // for std::cout

// Definition of user-defined function doPrint()
// doPrint() is the called function in this example
void doPrint()
{
    std::cout << "In doPrint()\n";
}

// Definition of user-defined function main()
int main()
{
    std::cout << "Starting main()\n";
    doPrint();                        // Interrupt main() by making a function call to doPrint().  main() is the caller.
    std::cout << "Ending main()\n";   // This statement is executed after doPrint() ends

    return 0;
```

```
}
```

This program produces the following output:

```
Starting main()
In doPrint()
Ending main()
```

This program begins execution at the top of function `main()`, and the first line to be executed prints `Starting main()`.

The second line in `main()` is a function call to the function `doPrint()`. We know it's a function call due to the trailing parentheses.

> **Warning**
>
> When calling a function, don't forget the parentheses `()` after the function's name. If you forget the parentheses, your program may not compile (and if it does, the function will not be called).

Because a function call was made, execution of statements in `main()` is suspended, and execution jumps to the top of called function `doPrint()`. The first (and only) line in `doPrint()` prints `In doPrint()`. When `doPrint()` terminates, execution returns back to the caller (`main()`) and continues from the point just beyond the function call. Consequently, the next statement executed in `main()` prints `Ending main()`.

Calling functions more than once

One useful thing about functions is that they can be called more than once. Here's a program that demonstrates this:

```cpp
#include <iostream> // for std::cout

void doPrint()
{
    std::cout << "In doPrint()\n";
}

// Definition of function main()
int main()
{
    std::cout << "Starting main()\n";
    doPrint(); // doPrint() called for the first time
    doPrint(); // doPrint() called for the second time
    std::cout << "Ending main()\n";

    return 0;
}
```

This program produces the following output:

```
Starting main()
In doPrint()
In doPrint()
Ending main()
```

Since `doPrint()` gets called twice by `main()`, `doPrint()` executes twice, and `In doPrint()` gets printed twice (once for each call).

Functions can call functions that call other functions

You've already seen that function `main()` can call other functions (such as function `doPrint()` in the example above). The functions called by `main()` can also call other functions (and those functions can call functions too, etc...). In the following program, function `main()` calls function `doA()`, which calls function `doB()`:

```
#include <iostream> // for std::cout

void doB()
{
    std::cout << "In doB()\n";
}

void doA()
{
    std::cout << "Starting doA()\n";

    doB();

    std::cout << "Ending doA()\n";
}

// Definition of function main()
int main()
{
    std::cout << "Starting main()\n";

    doA();

    std::cout << "Ending main()\n";

    return 0;
}
```

This program produces the following output:

```
Starting main()
Starting doA()
In doB()
Ending doA()
Ending main()
```

Nested functions are not supported

A function whose definition is placed inside another function is a **nested function**. Unlike some other programming languages, in C++, functions cannot be nested. The following program is not legal:

```
#include <iostream>

int main()
{
    void foo() // Illegal: this function is nested inside function
main()
    {
```

```
        std::cout << "foo!\n";
    }

    foo(); // function call to foo()

    return 0;
}
```

The proper way to write the above program is:

```
#include <iostream>

void foo() // no longer inside of main()
{
    std::cout << "foo!\n";
}

int main()
{
    foo();

    return 0;
}
```

> **Nomenclature**
>
> "foo" is a meaningless word that is often used as a placeholder name for a function or variable when the name is unimportant to the demonstration of some concept. Such words are called metasyntactic variables (though in common language they're often called "placeholder names" since nobody can remember the term "metasyntactic variable"). Other common metasyntactic variables in C++ include "bar", "baz", and 3-letter words that end in "oo", such as "goo", "moo", and "boo").
>
> For those interested in etymology (how words evolve), RFC 3092 is an interesting read.

Quiz time

In a function definition, what are the curly braces and statements in-between called?

> **Show Solution**
>
> The function body

What does the following program print? Do not compile this program, just trace the code yourself.

```
#include <iostream> // for std::cout

void doB()
{
    std::cout << "In doB()\n";
}

void doA()
{
    std::cout << "In doA()\n";
```

```
    doB();
}

// Definition of function main()
int main()
{
    std::cout << "Starting main()\n";

    doA();
    doB();

    std::cout << "Ending main()\n";

    return 0;
}
```

> **Show Solution**
>
> ```
> Starting main()
> In doA()
> In doB()
> In doB()
> Ending main()
> ```

02.02 — Function return values (value-returning functions)

Consider the following program:

```
#include <iostream>

int main()
{
    // get a value from the user
    std::cout << "Enter an integer: ";
    int num{};
    std::cin >> num;

    // print the value doubled
    std::cout << num << " doubled is: " << num * 2 << '\n';

    return 0;
}
```

This program is composed of two conceptual parts: First, we get a value from the user. Then we tell the user what double that value is.

Although this program is trivial enough that we don't need to break it into multiple functions, what if we wanted to? Getting an integer value from the user is a well-defined job that we want our program to do, so it would make a good candidate for a function.

So let's write a program to do this:

```
// This program doesn't work
#include <iostream>
```

```
void getValueFromUser()
{
    std::cout << "Enter an integer: ";
    int input{};
    std::cin >> input;
}

int main()
{
    getValueFromUser(); // Ask user for input

    int num{}; // How do we get the value from getValueFromUser()
and use it to initialize this variable?

    std::cout << num << " doubled is: " << num * 2 << '\n';

    return 0;
}
```

While this program is a good attempt at a solution, it doesn't quite work.

When function `getValueFromUser` is called, the user is asked to enter an integer as expected. But the value they enter is lost when `getValueFromUser` terminates and control returns to `main`. Variable `num` never gets initialized with the value the user entered, and so the program always prints the answer `0`.

What we're missing is some way for `getValueFromUser` to return the value the user entered back to `main` so that `main` can make use of that data.

Return values

When you write a user-defined function, you get to determine whether your function will return a value back to the caller or not. To return a value back to the caller, two things are needed.

First, your function has to indicate what type of value will be returned. This is done by setting the function's **return type**, which is the type that is defined before the function's name. In the example above, function `getValueFromUser` has a return type of `void` (meaning no value will be returned to the caller), and function `main` has a return type of `int` (meaning a value of type `int` will be returned to the caller). Note that this doesn't determine what specific value is returned – it only determines what *type* of value will be returned.

> **Related content**
>
> We explore functions that return `void` further in the next lesson (2.3 – Void functions (non-value returning functions)).

Second, inside the function that will return a value, we use a **return statement** to indicate the specific value being returned to the caller. The return statement consists of the `return` keyword, followed by an expression (sometimes called the **return expression**), ending with a semicolon.

When the return statement is executed:

- The return expression is evaluated to produce a value.
- The value produced by the return expression is copied back to the caller. This copy is called the **return value** of the function.
- The function exits, and control returns to the caller.

The process of returning a copied value back to the caller is named **return by value**.

> **Nomenclature**
>
> The return expression produces the value to be returned. The return value is a copy of that value.

A value-returning function will return a value each time it is called.

Let's take a look at a simple function that returns an integer value, and a sample program that calls it:

```
#include <iostream>

// int is the return type
// A return type of int means the function will return some integer value to the caller (the specific value is not specified here)
int returnFive()
{
    // the return statement provides the value that will be returned
    return 5; // return the value 5 back to the caller
}

int main()
{
    std::cout << returnFive() << '\n'; // prints 5
    std::cout << returnFive() + 2 << '\n'; // prints 7

    returnFive(); // okay: the value 5 is returned, but is ignored since main() doesn't do anything with it

    return 0;
}
```

When run, this program prints:

```
5
7
```

Execution starts at the top of `main`. In the first statement, the function call to `returnFive()` is evaluated, which results in function `returnFive()` being called. The return expression `5` is evaluated to produce the value `5`, which is returned back to the caller and printed to the console via `std::cout`.

In the second function call, the function call to `returnFive` is evaluated, which results in function `returnFive` being called again. Function `returnFive` returns the value of `5` back to the caller. The expression `5 + 2` is evaluated to produce the result `7`, which is then printed to the console via `std::cout`.

In the third statement, function `returnFive` is called again, resulting in the value `5` being returned back to the caller. However, function `main` does nothing with the return value, so nothing further happens (the return value is ignored).

Note: Return values will not be printed unless the caller sends them to the console via `std::cout`. In the last case above, the return value is not sent to `std::cout`, so nothing is printed.

> **Tip**
>
> When a called function returns a value, the caller may decide to use that value in an expression or statement (e.g. by using it to initialize a variable, or sending it to `std::cout`) or ignore it (by doing nothing else). If the caller ignores the return value, it is discarded (nothing is done with it).

Fixing our challenge program

With this in mind, we can fix the program we presented at the top of the lesson:

```
#include <iostream>

int getValueFromUser() // this function now returns an integer value
{
    std::cout << "Enter an integer: ";
    int input{};
    std::cin >> input;

    return input; // return the value the user entered back to the
caller
}

int main()
{
    int num { getValueFromUser() }; // initialize num with the
return value of getValueFromUser()

    std::cout << num << " doubled is: " << num * 2 << '\n';

    return 0;
}
```

When this program executes, the first statement in `main` will create an `int` variable named `num`. When the program goes to initialize `num`, it will see that there is a function call to `getValueFromUser()`, so it will go execute that function. Function `getValueFromUser`, asks the user to enter a value, and then it returns that value back to the caller (`main()`). This return value is used as the initialization value for variable `num`. `num` can then be used as many times as needed within `main()`.

> **Tip**
>
> If you need to use the return value of a function call more than once, initialize a variable with the return value, and then use that variable as many times as needed.

Compile this program yourself and run it a few times to prove to yourself that it works.

Revisiting main()

You now have the conceptual tools to understand how the `main()` function actually works. When the program is executed, the operating system makes a function call to `main()`. Execution then jumps to the top of `main()`. The statements in `main()` are executed sequentially. Finally, `main()` returns an integer value (usually `0`), and your program terminates.

In C++, there are two special requirements for `main()`:

- `main()` is required to return an `int`.
- Explicit function calls to `main()` are disallowed.

```
void foo()
{
    main(); // Compile error: main not allowed to be called explicitly
}

void main() // Compile error: main not allowed to have non-int return type
{
    foo();
}
```

> **Key insight**
>
> C does allow `main()` to be called explicitly, so some C++ compilers will allow this for compatibility reasons.

For now, you should also define your `main()` function at the bottom of your code file, below other functions, and avoid calling it explicitly.

> **For advanced readers**
>
> It is a common misconception that `main` is always the first function that executes.
>
> Global variables are initialized prior to the execution of `main`. If the initializer for such a variable invokes a function, then that function will execute prior to `main`. We discuss global variables in lesson 7.4 – Introduction to global variables.

Status codes

You may be wondering why we return 0 from `main()`, and when we might return something else.

The return value from `main()` is sometimes called a **status code** (or less commonly, an **exit code**, or rarely a **return code**). The status code is used to signal whether your program was successful or not.

By convention, a status code of `0` means the program ran normally (meaning the program executed and behaved as expected).

> **Best practice**
>
> Your `main` function should return the value `0` if the program ran normally.

A non-zero status code is often used to indicate some kind of failure (and while this works fine on most operating systems, strictly speaking, it's not guaranteed to be portable).

> **For advanced readers**
>
> The C++ standard only defines the meaning of 3 status codes: `0`, `EXIT_SUCCESS`, and `EXIT_FAILURE`. `0` and `EXIT_SUCCESS` both mean the program executed successfully. `EXIT_FAILURE` means the program did not execute successfully.
>
> `EXIT_SUCCESS` and `EXIT_FAILURE` are preprocessor macros defined in the `<cstdlib>` header:
>
> ```
> #include <cstdlib> // for EXIT_SUCCESS and EXIT_FAILURE
>
> int main()
> {
>  return EXIT_SUCCESS;
> }
> ```
>
> If you want to maximize portability, you should only use `0` or `EXIT_SUCCESS` to indicate a successful termination, or `EXIT_FAILURE` to indicate an unsuccessful termination.
>
> We cover the preprocessor and preprocessor macros in lesson 2.10 – Introduction to the preprocessor.

> **As an aside…**
>
> The status code is passed back to the operating system. The OS will typically make the status code available to whichever program launched the program returning the status code. This provides a crude mechanism for any program launching another program to determine whether the launched program ran successfully or not.

A value-returning function that does not return a value will produce undefined behavior

A function that returns a value is called a **value-returning function**. A function is value-returning if the return type is anything other than `void`.

A value-returning function *must* return a value of that type (using a return statement), otherwise undefined behavior will result.

> **Related content**
>
> We discuss undefined behavior in lesson 1.6 – Uninitialized variables and undefined behavior.

Here's an example of a function that produces undefined behavior:

```
#include <iostream>

int getValueFromUserUB() // this function returns an integer value
{
    std::cout << "Enter an integer: ";
```

```
    int input{};
    std::cin >> input;

    // note: no return statement
}

int main()
{
    int num { getValueFromUserUB() }; // initialize num with the return value of getValueFromUserUB()

    std::cout << num << " doubled is: " << num * 2 << '\n';

    return 0;
}
```

A modern compiler should generate a warning because `getValueFromUserUB` is defined as returning an `int` but no return statement is provided. Running such a program would produce undefined behavior, because `getValueFromUserUB()` is a value-returning function that does not return a value.

In most cases, compilers will detect if you've forgotten to return a value. However, in some complicated cases, the compiler may not be able to properly determine whether your function returns a value or not in all cases, so you should not rely on this.

> **Best practice**
>
> Make sure your functions with non-void return types return a value in all cases.
>
> Failure to return a value from a value-returning function will cause undefined behavior.

Function main will implicitly return 0 if no return statement is provided

The only exception to the rule that a value-returning function must return a value via a return statement is for function `main()`. The function `main()` will implicitly return the value `0` if no return statement is provided. That said, it is best practice to explicitly return a value from `main`, both to show your intent, and for consistency with other functions (which will exhibit undefined behavior if a return value is not specified).

Functions can only return a single value

A value-returning function can only return a single value back to the caller each time it is called.

Note that the value provided in a return statement doesn't need to be literal – it can be the result of any valid expression, including a variable or even a call to another function that returns a value. In the `getValueFromUser()` example above, we returned a variable `input`, which held the number the user input.

There are various ways to work around the limitation of functions only being able to return a single value, which we'll cover in future lessons.

The function author can decide what the return value means

The meaning of the value returned by a function is determined by the function's author. Some functions use return values as status codes, to indicate whether they succeeded or failed. Other functions return a calculated or selected value. Other functions return nothing (we'll see examples of these in the next lesson).

Because of the wide variety of possibilities here, it's a good idea to document your function with a comment indicating what the return values mean. For example:

```
// Function asks user to enter a value
// Return value is the integer entered by the user from the keyboard
int getValueFromUser()
{
    std::cout << "Enter an integer: ";
    int input{};
    std::cin >> input;

    return input; // return the value the user entered back to the
caller
}
```

Reusing functions

Now we can illustrate a good case for function reuse. Consider the following program:

```
#include <iostream>

int main()
{
    int x{};
    std::cout << "Enter an integer: ";
    std::cin >> x;

    int y{};
    std::cout << "Enter an integer: ";
    std::cin >> y;

    std::cout << x << " + " << y << " = " << x + y << '\n';

    return 0;
}
```

While this program works, it's a little redundant. In fact, this program violates one of the central tenets of good programming: **Don't Repeat Yourself** (often abbreviated **DRY**).

Why is repeated code bad? If we wanted to change the text "Enter an integer:" to something else, we'd have to update it in two locations. And what if we wanted to initialize 10 variables instead of 2? That

would be a lot of redundant code (making our programs longer and harder to understand), and a lot of room for typos to creep in.

Let's update this program to use our `getValueFromUser` function that we developed above:

```
#include <iostream>

int getValueFromUser()
{
    std::cout << "Enter an integer: ";
    int input{};
    std::cin >> input;

    return input;
}

int main()
{
    int x{ getValueFromUser() }; // first call to getValueFromUser
    int y{ getValueFromUser() }; // second call to getValueFromUser

    std::cout << x << " + " << y << " = " << x + y << '\n';

    return 0;
}
```

This program produces the following output:

```
Enter an integer: 5
Enter an integer: 7
5 + 7 = 12
```

In this program, we call `getValueFromUser` twice, once to initialize variable `x`, and once to initialize variable `y`. That saves us from duplicating the code to get user input, and reduces the odds of making a mistake. Once we know `getValueFromUser` works, we can call it as many times as we desire.

This is the essence of modular programming: the ability to write a function, test it, ensure that it works, and then know that we can reuse it as many times as we want and it will continue to work (so long as we don't modify the function – at which point we'll have to retest it).

> **Best practice**
>
> Follow DRY: "Don't repeat yourself". If you need to do something more than once, consider how to modify your code to remove as much redundancy as possible. Variables can be used to store the results of calculations that need to be used more than once (so we don't have to repeat the calculation). Functions can be

---

02.03 — Void functions (non-value returning functions)

In a prior lesson (2.1 – Introduction to functions), we indicated that the syntax for a function definition looks like this:

```
        returnType identifier() // identifier replaced with the name of your
function
        {
        // Your code here
        }
```

Although we showed examples of functions that had return-type `void`, we did not discuss what this meant. In this lesson, we'll explore functions with a return type of `void`.

Void return values

Functions are not required to return a value back to the caller. To tell the compiler that a function does not return a value, a return type of **void** is used. For example:

```
        #include <iostream>

        // void means the function does not return a value to the caller
        void printHi()
        {
            std::cout << "Hi" << '\n';

            // This function does not return a value so no return statement
is needed
        }

        int main()
        {
            printHi(); // okay: function printHi() is called, no value is
returned

            return 0;
        }
```

In the above example, the `printHi` function has a useful behavior (it prints "Hi") but it doesn't need to return anything back to the caller. Therefore, `printHi` is given a `void` return type.

When `main` calls `printHi`, the code in `printHi` executes, and "Hi" is printed. At the end of `printHi`, control returns to `main` and the program proceeds.

A function that does not return a value is called a **non-value returning function** (or a **void function**).

Void functions don't need a return statement

A void function will automatically return to the caller at the end of the function. No return statement is required.

A return statement (with no return value) can be used in a void function – such a statement will cause the function to return to the caller at the point where the return statement is executed. This is the same thing that happens at the end of the function anyway. Consequently, putting an empty return statement at the end of a void function is redundant:

```
#include <iostream>

// void means the function does not return a value to the caller
void printHi()
{
    std::cout << "Hi" << '\n';

    return; // tell compiler to return to the caller -- this is
redundant since the return will happen at the end of the function
anyway!
} // function will return to caller here

int main()
{
    printHi();

    return 0;
}
```

> **Best practice**
>
> Do not put a return statement at the end of a non-value returning function.

Void functions can't be used in expression that require a value

Some types of expressions require values. For example:

```
#include <iostream>

int main()
{
    std::cout << 5; // ok: 5 is a literal value that we're sending
to the console to be printed
    std::cout << ;  // compile error: no value provided

    return 0;
}
```

In the above program, the value to be printed needs to be provided on the right-side of the `std::cout <<`. If no value is provided, the compiler will produce a syntax error. Since the second call to `std::cout` does not provide a value to be printed, this causes an error.

Now consider the following program:

```
#include <iostream>

// void means the function does not return a value to the caller
void printHi()
{
    std::cout << "Hi" << '\n';
}

int main()
{
    printHi(); // okay: function printHi() is called, no value is
returned
```

```
    std::cout << printHi(); // compile error

    return 0;
}
```

The first call to `printHi()` is called in a context that does not require a value. Since the function doesn't return a value, this is fine.

The second function call to function `printHi()` won't even compile. Function `printHi` has a `void` return type, meaning it doesn't return a value. However, this statement is trying to send the return value of `printHi` to `std::cout` to be printed. `std::cout` doesn't know how to handle this (what value would it output?). Consequently, the compiler will flag this as an error. You'll need to comment out this line of code in order to make your code compile.

> **Tip**
>
> Some statements require values to be provided, and others don't.
>
> When we have a statement that consists of just a function call (e.g. the first `printHi()` in the above example), we're calling a function for its behavior, not its return value. In this case, we can call either a non-value returning function, or we can call a value-returning function and just ignore the return value.
>
> When we call a function in a context that requires a value (e.g. `std::cout`), a value must be provided. In such a context, we can only call value-returning functions.

```
#include <iostream>

// Function that does not return a value
void returnNothing()
{
}

// Function that returns a value
int returnFive()
{
    return 5;
}

int main()
{
    // When calling a function by itself, no value is required
    returnNothing(); // ok: we can call a function that does not
return a value
    returnFive();    // ok: we can call a function that returns a
value, and ignore that return value

    // When calling a function in a context that requires a value
(like std::cout)
    std::cout << returnFive();    // ok: we can call a function that
returns a value, and the value will be used
    std::cout << returnNothing(); // compile error: we can't call a
function that returns void in this context

    return 0;
```

```
}
```

Returning a value from a void function is a compile error

Trying to return a value from a non-value returning function will result in a compilation error:

```
void printHi() // This function is non-value returning
{
    std::cout << "In printHi()" << '\n';

    return 5; // compile error: we're trying to return a value
}
```

Quiz time

Inspect the following programs and state what they output, or whether they will not compile.

1a)

```
#include <iostream>

void printA()
{
    std::cout << "A\n";
}

void printB()
{
    std::cout << "B\n";
}

int main()
{
    printA();
    printB();

    return 0;
}
```

This program prints the letters A and B on separate lines.

1b)

```
#include <iostream>

void printA()
{
    std::cout << "A\n";
}

int main()
{
    std::cout << printA() << '\n';

    return 0;
```

```
}
```

This program does not compile. Function `printA()` returns `void`, which can't be sent to `std::cout` to be printed. This will produce a compile error.

02.04 — Introduction to function parameters and arguments

In the previous lesson, we learned that we could have a function return a value back to the function's caller. We used that to create a modular *getValueFromUser* function that we used in this program:

```
#include <iostream>

int getValueFromUser()
{
    std::cout << "Enter an integer: ";
    int input{};
    std::cin >> input;

    return input;
}

int main()
{
    int num { getValueFromUser() };

    std::cout << num << " doubled is: " << num * 2 << '\n';

    return 0;
}
```

However, what if we wanted to put the output line into its own function as well? You might try something like this:

```
#include <iostream>

int getValueFromUser()
{
    std::cout << "Enter an integer: ";
    int input{};
    std::cin >> input;

    return input;
}

// This function won't compile
void printDouble()
{
    std::cout << num << " doubled is: " << num * 2 << '\n';
}

int main()
{
    int num { getValueFromUser() };
```

```
    printDouble();

    return 0;
}
```

This won't compile, because function *printDouble* doesn't know what identifier *num* is. You might try defining num as a variable inside function printDouble():

```
void printDouble()
{
    int num{}; // we added this line
    std::cout << num << " doubled is: " << num * 2 << '\n';
}
```

While this addresses the compiler error and makes the program compile-able, the program still doesn't work correctly (it always prints "0 doubled is: 0"). The core of the problem here is that function *printDouble* doesn't have a way to access the value the user entered.

We need some way to pass the value of variable *num* to function *printDouble* so that *printDouble* can use that value in the function body.

Function parameters and arguments

In many cases, it is useful to be able to pass information *to* a function being called, so that the function has data to work with. For example, if we wanted to write a function to add two numbers, we need some way to tell the function which two numbers to add when we call it. Otherwise, how would the function know what to add? We do that via function parameters and arguments.

A **function parameter** is a variable used in the header of a function. Function parameters work almost identically to variables defined inside the function, but with one difference: they are initialized with a value provided by the caller of the function.

Function parameters are defined in the function header by placing them in between the parenthesis after the function name, with multiple parameters being separated by commas.

Here are some examples of functions with different numbers of parameters:

```
// This function takes no parameters
// It does not rely on the caller for anything
void doPrint()
{
    std::cout << "In doPrint()\n";
}

// This function takes one integer parameter named x
// The caller will supply the value of x
void printValue(int x)
{
    std::cout << x << '\n';
}

// This function has two integer parameters, one named x, and one
```

```
named y
        // The caller will supply the value of both x and y
        int add(int x, int y)
        {
            return x + y;
        }
```

An **argument** is a value that is passed *from* the caller *to* the function when a function call is made:

```
        doPrint(); // this call has no arguments
        printValue(6); // 6 is the argument passed to function printValue()
        add(2, 3); // 2 and 3 are the arguments passed to function add()
```

Note that multiple arguments are also separated by commas.

How parameters and arguments work together

When a function is called, all of the parameters of the function are created as variables, and the value of each of the arguments is *copied* into the matching parameter (using copy initialization). This process is called **pass by value**. Function parameters that utilize pass by value are called **value parameters**.

For example:

```
        #include <iostream>

        // This function has two integer parameters, one named x, and one
named y
        // The values of x and y are passed in by the caller
        void printValues(int x, int y)
        {
            std::cout << x << '\n';
            std::cout << y << '\n';
        }

        int main()
        {
            printValues(6, 7); // This function call has two arguments, 6
and 7

            return 0;
        }
```

When function *printValues* is called with arguments *6* and *7*, *printValues*'s parameter *x* is created and initialized with the value of *6*, and *printValues*'s parameter *y* is created and initialized with the value of *7*.

This results in the output:

```
6
7
```

Note that the number of arguments must generally match the number of function parameters, or the compiler will throw an error. The argument passed to a function can be any valid expression (as the

argument is essentially just an initializer for the parameter, and initializers can be any valid expression).

Fixing our challenge program

We now have the tool we need to fix the program we presented at the top of the lesson:

```
#include <iostream>

int getValueFromUser()
{
    std::cout << "Enter an integer: ";
    int input{};
    std::cin >> input;

    return input;
}

void printDouble(int value) // This function now has an integer parameter
{
    std::cout << value << " doubled is: " << value * 2 << '\n';
}

int main()
{
    int num { getValueFromUser() };

    printDouble(num);

    return 0;
}
```

In this program, variable *num* is first initialized with the value entered by the user. Then, function *printDouble* is called, and the value of argument *num* is copied into the *value* parameter of function *printDouble*. Function *printDouble* then uses the value of parameter *value*.

Using return values as arguments

In the above problem, we can see that variable *num* is only used once, to transport the return value of function *getValueFromUser* to the argument of the call to function *printDouble*.

We can simplify the above example slightly as follows:

```
#include <iostream>

int getValueFromUser()
{
    std::cout << "Enter an integer: ";
    int input{};
    std::cin >> input;

    return input;
}
```

```
void printDouble(int value)
{
    std::cout << value << " doubled is: " << value * 2 << '\n';
}

int main()
{
    printDouble(getValueFromUser());

    return 0;
}
```

Now, we're using the return value of function *getValueFromUser* directly as an argument to function *printDouble*!

Although this program is more concise (and makes it clear that the value read by the user will be used for nothing else), you may also find this "compact syntax" a bit hard to read. If you're more comfortable sticking with the version that uses the variable instead, that's fine.

How parameters and return values work together

By using both parameters and a return value, we can create functions that take data as input, do some calculation with it, and return the value to the caller.

Here is an example of a very simple function that adds two numbers together and returns the result to the caller:

```
#include <iostream>

// add() takes two integers as parameters, and returns the result of
their sum
// The values of x and y are determined by the function that calls
add()
int add(int x, int y)
{
    return x + y;
}

// main takes no parameters
int main()
{
    std::cout << add(4, 5) << '\n'; // Arguments 4 and 5 are passed
to function add()
    return 0;
}
```

Execution starts at the top of *main*. When `add(4, 5)` is evaluated, function *add* is called, with parameter *x* being initialized with value *4*, and parameter *y* being initialized with value *5*.

The *return statement* in function *add* evaluates *x + y* to produce the value *9*, which is then returned back to *main*. This value of *9* is then sent to `std::cout` to be printed on the console.

Output:

More examples

Let's take a look at some more function calls:

```
#include <iostream>

int add(int x, int y)
{
    return x + y;
}

int multiply(int z, int w)
{
    return z * w;
}

int main()
{
    std::cout << add(4, 5) << '\n'; // within add() x=4, y=5, so
x+y=9
    std::cout << add(1 + 2, 3 * 4) << '\n'; // within add() x=3,
y=12, so x+y=15

    int a{ 5 };
    std::cout << add(a, a) << '\n'; // evaluates (5 + 5)

    std::cout << add(1, multiply(2, 3)) << '\n'; // evaluates 1 + (2
* 3)
    std::cout << add(1, add(2, 3)) << '\n'; // evaluates 1 + (2 + 3)

    return 0;
}
```

This program produces the output:

```
9
15
10
7
6
```

The first statement is straightforward.

In the second statement, the arguments are expressions that get evaluated before being passed. In this case, *1 + 2* evaluates to *3*, so *3* is copied to parameter *x*. *3* 4* evaluates to *12*, so *12* is copied to parameter *y*. *add(3, 12)* resolves to *15*.

The next pair of statements is relatively easy as well:

```
int a{ 5 };
    std::cout << add(a, a) << '\n'; // evaluates (5 + 5)
```

In this case, *add()* is called where the value of *a* is copied into both parameters *x* and *y*. Since *a* has value *5*, *add(a, a) = add(5, 5)*, which resolves to value *10*.

Let's take a look at the first tricky statement in the bunch:

```
std::cout << add(1, multiply(2, 3)) << '\n'; // evaluates 1 + (2 * 3)
```

When the function *add* is executed, the program needs to determine what the values for parameters *x* and *y* are. *x* is simple since we just passed it the integer *1*. To get a value for parameter *y*, it needs to evaluate *multiply(2, 3)* first. The program calls *multiply* and initializes *z = 2* and *w = 3*, so *multiply(2, 3)* returns the integer value *6*. That return value of *6* can now be used to initialize the *y* parameter of the *add* function. *add(1, 6)* returns the integer *7*, which is then passed to std::cout for printing.

Put less verbosely: *add(1, multiply(2, 3))* evaluates to *add(1, 6)* evaluates to *7*

The following statement looks tricky because one of the arguments given to *add* is another call to *add*.

```
std::cout << add(1, add(2, 3)) << '\n'; // evaluates 1 + (2 + 3)
```

But this case works exactly the same as the prior case. add(2, 3) resolves first, resulting in the return value of *5*. Now it can resolve add(1, 5), which evaluates to the value *6*, which is passed to std::cout for printing.

Less verbosely: *add(1, add(2, 3))* evaluates to *add(1, 5)* => evaluates to *6*

Unreferenced parameters and unnamed parameters

In certain cases, you will encounter functions that have parameters that are not used in the body of the function. These are called **unreferenced parameters**.

As a trivial example:

```
void doSomething(int count) // warning: unreferenced parameter count
{
    // This function used to do something with count but it is not used any longer
}

int main()
{
    doSomething(4);

    return 0;
}
```

Just like with unused local variables, your compiler will probably warn that variable `count` has been defined but not used.

In a function definition, the name of a function parameter is optional. Therefore, in cases where a function parameter needs to exist but is not used in the body of the function, you can simply omit the name. A parameter without a name is called an **unnamed parameter**:

```
void doSomething(int) // ok: unnamed parameter will not generate warning
```

```
{
}
```

The Google C++ style guide recommends using a comment to document what the unnamed parameter was:

```
void doSomething(int /*count*/)
{
}
```

> **Author's note**
>
> You're probably wondering why we'd write a function that has a parameter whose value isn't used. This happens most often in cases similar to the following:
>
> 1. Let's say we have a function with a single parameter. Later, the function is updated in some way, and the value of the parameter is no longer needed. If the now-unused function parameter were simply removed, then every existing call to the function would break (because the function call would be supplying more arguments than the function could accept). This would require us to find every call to the function and remove the unneeded argument. This might be a lot of work (and require a lot of retesting). It also might not even be possible (in cases where we did not control all of the code calling the function). So instead, we might leave the parameter as it is, and just have it do nothing.

> **For advanced readers**
>
> Other cases where this occurs:
>
> 1. Operators `++` and `--` have prefix and postfix variants (e.g. `++foo` vs `foo++`). An unreferenced function parameter is used to differentiate whether an overload of such an operator is for the prefix or postfix case. We cover this in lesson 21.8 – Overloading the increment and decrement operators.
> 2. When we need to determine something from the type (rather than the value) of a type template parameter.

> **Author's note**
>
> If unnamed parameters still don't make sense to you yet, don't worry. We'll encounter them again in future lessons, when we have more context to explain when they are useful.

> **Best practice**
>
> When a function parameter exists but is not used in the body of the function, do not give it a name. You can optionally put a name inside a comment.

Conclusion

Function parameters and return values are the key mechanisms by which functions can be written in a reusable way, as it allows us to write functions that can perform tasks and return retrieved or calculated results back to the caller without knowing what the specific inputs or outputs are ahead of time.

Quiz time

Question #1

What's wrong with this program fragment?

```
#include <iostream>

void multiply(int x, int y)
{
    return x * y;
}

int main()
{
    std::cout << multiply(4, 5) << '\n';

    return 0;
}
```

`multiply()` has a return type of `void`, meaning it is a non-value returning function. Since the function is trying to return a value (via a return statement), this function will produce a compiler error. The return type should be `int`.

Question #2

What two things are wrong with this program fragment?

```
#include <iostream>

int multiply(int x, int y)
{
    int product { x * y };
}

int main()
{
    std::cout << multiply(4) << '\n';

    return 0;
}
```

Problem 1: `main()` passes one argument to `multiply()`, but `multiply()` requires two arguments. Problem 2: `multiply()` doesn't have a return statement.

Question #3

What value does the following program print?

```
#include <iostream>

int add(int x, int y, int z)
{
    return x + y + z;
}

int multiply(int x, int y)
{
    return x * y;
```

```
}

int main()
{
    std::cout << multiply(add(1, 2, 3), 4) << '\n';

    return 0;
}
```

`multiply` is called where x = `add(1, 2, 3)

02.05 — Introduction to local scope

Local variables

Variables defined inside the body of a function are called **local variables** (as opposed to *global variables*, which we'll discuss in a future chapter):

```
int add(int x, int y)
{
    int z{ x + y }; // z is a local variable

    return z;
}
```

Function parameters are also generally considered to be local variables, and we will include them as such:

```
int add(int x, int y) // function parameters x and y are local variables
{
    int z{ x + y };

    return z;
}
```

In this lesson, we'll take a look at some properties of local variables in more detail.

Local variable lifetime

In lesson 1.3 – Introduction to objects and variables, we discussed how a variable definition such as `int x;` causes the variable to be instantiated when this statement is executed. Function parameters are created and initialized when the function is entered, and variables within the function body are created and initialized at the point of definition.

For example:

```
int add(int x, int y) // x and y created and initialized here
{
    int z{ x + y };   // z created and initialized here
```

```
    return z;
}
```

The natural follow-up question is, “so when is an instantiated variable destroyed?”. Local variables are destroyed in the opposite order of creation at the end of the set of curly braces in which it is defined (or for a function parameter, at the end of the function).

```
int add(int x, int y)
{
    int z{ x + y };

    return z;
} // z, y, and x destroyed here
```

Much like a person’s lifetime is defined to be the time between their birth and death, an object’s **lifetime** is defined to be the time between its creation and destruction. Note that variable creation and destruction happen when the program is running (called runtime), not at compile time. Therefore, lifetime is a runtime property.

> **For advanced readers**
>
> The above rules around creation, initialization, and destruction are guarantees. That is, objects must be created and initialized no later than the point of definition, and destroyed no earlier than the end of the set of the curly braces in which they are defined (or, for function parameters, at the end of the function).
>
> In actuality, the C++ specification gives compilers a lot of flexibility to determine when local variables are created and destroyed. Objects may be created earlier, or destroyed later for optimization purposes. Most often, local variables are created when the function is entered, and destroyed in the opposite order of creation when the function is exited. We’ll discuss this in more detail in a future lesson, when we talk about the call stack.

Here’s a slightly more complex program demonstrating the lifetime of a variable named `x`:

```
#include <iostream>

void doSomething()
{
    std::cout << "Hello!\n";
}

int main()
{
    int x{ 0 };    // x's lifetime begins here

    doSomething(); // x is still alive during this function call

    return 0;
} // x's lifetime ends here
```

In the above program, the lifetime of `x` runs from the point of definition to the end of function `main`. This includes the time spent during the execution of function `doSomething`.

What happens when an object is destroyed?

In most cases, nothing. The destroyed object simply becomes invalid.

> **For advanced readers**
>
> If the object is a class type object, prior to destruction, a special function called a destructor is invoked. In many cases, the destructor does nothing, in which case no cost is incurred. We introduce destructors in lesson 15.4 – Introduction to destructors.

Any use of an object after it has been destroyed will result in undefined behavior.

At some point after destruction, the memory used by the object will be **deallocated** (freed up for reuse).

Local scope (block scope)

An identifier's **scope** determines where the identifier can be seen and used within the source code. When an identifier can be seen and used, we say it is **in scope**. When an identifier can not be seen, we can not use it, and we say it is **out of scope**. Scope is a compile-time property, and trying to use an identifier when it is not in scope will result in a compile error.

The identifier of a local variable has local scope. An identifier with **local scope** (technically called **block scope**) is usable from the point of definition to the end of the innermost pair of curly braces containing the identifier (or for function parameters, at the end of the function). This ensures local variables cannot be used before the point of definition (even if the compiler opts to create them before then) or after they are destroyed. Local variables defined in one function are also not in scope in other functions that are called.

Here's a program demonstrating the scope of a variable named `x`:

```
#include <iostream>

// x is not in scope anywhere in this function
void doSomething()
{
    std::cout << "Hello!\n";
}

int main()
{
    // x can not be used here because it's not in scope yet

    int x{ 0 }; // x enters scope here and can now be used within
this function

    doSomething();

    return 0;
} // x goes out of scope here and can no longer be used
```

In the above program, variable `x` enters scope at the point of definition. `x` goes out of scope at the end of the innermost pair of curly braces containing the identifier, which is the closing curly brace of function `main()`. Note that variable `x` is not in scope anywhere inside of function `doSomething`. The fact that function `main` calls function `doSomething` is irrelevant in this context.

“Out of scope” vs “going out of scope”

The terms “out of scope” and “going out of scope” can be confusing to new programmers.

An identifier is out of scope anywhere it cannot be accessed within the code. In the example above, the identifier `x` is in scope from its point of definition to the end of the `main` function. The identifier `x` is out of scope outside of that code region.

The term “going out of scope” is typically applied to objects rather than identifiers. We say an object goes out of scope at the end of the scope (the end curly brace) in which the object was instantiated. In the example above, the object named `x` goes out of scope at the end of the function `main`.

A local variable’s lifetime ends at the point where it goes out of scope, so local variables are destroyed at this point.

Note that not all types of variables are destroyed when they go out of scope. We’ll see examples of these in future lessons.

Another example

Here’s a slightly more complex example. Remember, lifetime is a runtime property, and scope is a compile-time property, so although we are talking about both in the same program, they are enforced at different points.

```
#include <iostream>

int add(int x, int y) // x and y are created and enter scope here
{
    // x and y are usable only within add()
    return x + y;
} // y and x go out of scope and are destroyed here

int main()
{
    int a{ 5 }; // a is created, initialized, and enters scope here
    int b{ 6 }; // b is created, initialized, and enters scope here

    // a and b are usable only within main()

    std::cout << add(a, b) << '\n'; // calls add(5, 6), where x=5
and y=6

    return 0;
} // b and a go out of scope and are destroyed here
```

Parameters `x` and `y` are created when the `add` function is called, can only be seen/used within function `add`, and are destroyed at the end of `add`. Variables `a` and `b` are created within function `main`, can only be seen/used within function `main`, and are destroyed at the end of `main`.

To enhance your understanding of how all this fits together, let's trace through this program in a little more detail. The following happens, in order:

- Execution starts at the top of `main`.
- `main` variable `a` is created and given value `5`.
- `main` variable `b` is created and given value `6`.
- Function `add` is called with argument values `5` and `6`.
- `add` parameters `x` and `y` are created and initialized with values `5` and `6` respectively.
- The expression `x + y` is evaluated to produce the value `11`.
- `add` copies the value `11` back to caller `main`.
- `add` parameters `y` and `x` are destroyed.
- `main` prints `11` to the console.
- `main` returns `0` to the operating system.
- `main` variables `b` and `a` are destroyed.

And we're done.

Note that if function `add` were to be called twice, parameters `x` and `y` would be created and destroyed twice – once for each call. In a program with lots of functions and function calls, variables are created and destroyed often.

Functional separation

In the above example, it's easy to see that variables `a` and `b` are different variables from `x` and `y`.

Now consider the following similar program:

```
#include <iostream>

int add(int x, int y) // add's x and y are created and enter scope here
{
    // add's x and y are visible/usable within this function only
    return x + y;
} // add's y and x go out of scope and are destroyed here

int main()
{
    int x{ 5 }; // main's x is created, initialized, and enters scope here
    int y{ 6 }; // main's y is created, initialized, and enters scope here

    // main's x and y are usable within this function only
    std::cout << add(x, y) << '\n'; // calls function add() with x=5 and y=6

    return 0;
} // main's y and x go out of scope and are destroyed here
```

In this example, all we've done is change the names of variables `a` and `b` inside of function `main` to `x` and `y`. This program compiles and runs identically, even though functions `main` and `add` both have variables named `x` and `y`. Why does this work?

First, we need to recognize that even though functions `main` and `add` both have variables named `x` and `y`, these variables are distinct. The `x` and `y` in function `main` have nothing to do with the `x` and `y` in function `add` – they just happen to share the same names.

Second, when inside of function `main`, the names `x` and `y` refer to main's locally scoped variables `x` and `y`. Those variables can only be seen (and used) inside of `main`. Similarly, when inside function `add`, the names `x` and `y` refer to function parameters `x` and `y`, which can only be seen (and used) inside of `add`.

In short, neither `add` nor `main` know that the other function has variables with the same names. Because the scopes don't overlap, it's always clear to the compiler which `x` and `y` are being referred to at any time.

> **Key insight**
>
> Names used for function parameters or variables declared in a function body are only visible within the function that declares them. This means local variables within a function can be named without regard for the names of variables in other functions. This helps keep functions independent.

We'll talk more about local scope, and other kinds of scope, in a future chapter.

Where to define local variables

In modern C++, the best practice is that local variables inside the function body should be defined as close to their first use as reasonable:

```
#include <iostream>

int main()
{
    std::cout << "Enter an integer: ";
    int x{};       // x defined here
    std::cin >> x; // and used here

    std::cout << "Enter another integer: ";
    int y{};       // y defined here
    std::cin >> y; // and used here

    int sum{ x + y }; // sum can be initialized with intended value
    std::cout << "The sum is: " << sum << '\n';

    return 0;
}
```

In the above example, each variable is defined just before it is first used. There's no need to be strict about this – if you prefer to swap lines 5 and 6, that's fine.

> **Best practice**
>
> Define your local variables as close to their first use as reasonable.

> **As an aside…**
>
> Due to the limitations of older, more primitive compilers, the C language used to require all local variables be defined at the top of the function. The equivalent C++ program using that style would look like this:
>
> ```
> #include <iostream>
>
> int main()
> {
>  int x{}, y{}, sum{}; // how are these used?
>
>  std::cout << "Enter an integer: ";
>  std::cin >> x;
>
>  std::cout << "Enter another integer: ";
>  std::cin >> y;
>
>  sum = x + y;
>  std::cout << "The sum is: " << sum << '\n';
>
>  return 0;
> }
> ```
>
> This style is suboptimal for several reasons:
>
> - The intended use of these variables isn't apparent at the point of definition. You have to scan through the entire function to determine where and how each variable is used.
> - The intended initialization value may not be available at the top of the function (e.g. we can't initialize `sum` to its intended value because we don't know the value of `x` and `y` yet).
> - There may be many lines between a variable's initializer and its first use. If we don't remember what value it was initialized with, we will have to scroll back to the top of the function, which is distracting.
>
> This restriction was lifted in the C99 language standard.

When to use function parameters vs Local variables

Because function parameters and local variable can both be used within the body of a function, new programmers sometimes struggle to understand when each should be used. A function parameter should be used when the caller will pass in the initialization value as an argument. A local variable should be used otherwise.

Using a function parameter when you should use a local variable leads to code looking like this:

```
#include <iostream>

int getValueFromUser(int val) // val is a function parameter
```

```
{
    std::cout << "Enter a value: ";
    std::cin >> val;
    return val;
}

int main()
{
    int x {};
    int num { getValueFromUser(x) }; // main must pass x as an
argument

    std::cout << "You entered " << num << '\n';

    return 0;
}
```

In the above example, `getValueFromUser()` has defined `val` as a function parameter. Because of this, `main()` must define `x` so that it has something to pass as an argument. However, the actual value of `x` is never used, and the value that `val` is initialized with is never used. Making the caller define and pass a variable that is never used adds needless complexity.

The correct way to write this would be as follows:

```
#include <iostream>

int getValueFromUser()
{
    int val {}; // val is a local variable
    std::cout << "Enter a value: ";
    std::cin >> val;
    return val;
}

int main()
{
    int num { getValueFromUser() }; // main does not need to pass
anything

    std::cout << "You entered " << num << '\n';

    return 0;
}
```

In this example, `val` is now a local variable. `main()` is now simpler because it does not need to define or pass a variable to call `getValueFromUser()`.

> **Best practice**
>
> When a variable is needed within a function:
>
> - Use a function parameter when the caller will pass in the initialization value for the variable as an argument.
> - Use a local variable otherwise.

Introduction to temporary objects

A **temporary object** (also sometimes called an **anonymous object**) is an unnamed object that is used to hold a value that is only needed for a short period of time. Temporary objects are generated by the compiler when they are needed.

There are many different ways that temporary values can be created, but here's a common one:

```
#include <iostream>

int getValueFromUser()
{
    std::cout << "Enter an integer: ";
    int input{};
    std::cin >> input;

    return input; // return the value of input back to the caller
}

int main()
{
    std::cout << getValueFromUser() << '\n'; // where does the
returned value get stored?

    return 0;
}
```

In the above program, the function `getValueFromUser()` returns the value stored in local variable `input` back to the caller

---

02.06 — Why functions are useful, and how to use them effectively

Now that we've covered what functions are and some of their basic capabilities, let's take a closer look at why they're useful.

Why use functions?

New programmers often ask, "Can't we just put all the code inside the *main* function?" For simple programs, you absolutely can. However, functions provide a number of benefits that make them extremely useful in programs of non-trivial length or complexity.

- **Organization** – As programs grow in complexity, having all the code live inside the main() function becomes increasingly complicated. A function is almost like a mini-program that we can write separately from the main program, without having to think about the rest of the program while we write it. This allows us to reduce a complicated program into smaller, more manageable chunks, which reduces the overall complexity of our program.
- **Reusability** – Once a function is written, it can be called multiple times from within the program. This avoids duplicated code ("Don't Repeat Yourself") and minimizes the probability of copy/paste errors. Functions can also be shared with other programs, reducing the amount of code that has to be written from scratch (and retested) each time.

- **Testing** – Because functions reduce code redundancy, there's less code to test in the first place. Also because functions are self-contained, once we've tested a function to ensure it works, we don't need to test it again unless we change it. This reduces the amount of code we have to test at one time, making it much easier to find bugs (or avoid them in the first place).
- **Extensibility** – When we need to extend our program to handle a case it didn't handle before, functions allow us to make the change in one place and have that change take effect every time the function is called.
- **Abstraction** – In order to use a function, you only need to know its name, inputs, outputs, and where it lives. You don't need to know how it works, or what other code it's dependent upon to use it. This lowers the amount of knowledge required to use other people's code (including everything in the standard library).

Effectively using functions

One of the biggest challenges new programmers encounter (besides learning the language) is understanding when and how to use functions effectively. Here are a few basic guidelines for writing functions:

- Groups of statements that appear more than once in a program should generally be made into a function. For example, if we're reading input from the user multiple times in the same way, that's a great candidate for a function. If we output something in the same way in multiple places, that's also a great candidate for a function.
- Code that has a well-defined set of inputs and outputs is a good candidate for a function, (particularly if it is complicated). For example, if we have a list of items that we want to sort, the code to do the sorting would make a great function, even if it's only done once. The input is the unsorted list, and the output is the sorted list. Another good prospective function would be code that simulates the roll of a 6-sided dice. Your current program might only use that in one place, but if you turn it into a function, it's ready to be reused if you later extend your program or in a future program.
- A function should generally perform one (and only one) task.
- When a function becomes too long, too complicated, or hard to understand, it can be split into multiple sub-functions. This is called **refactoring**. We talk more about refactoring in lesson 3.10 – Finding issues before they become problems.

Typically, when learning C++, you will write a lot of programs that involve 3 subtasks:

1. Reading inputs from the user
2. Calculating a value from the inputs
3. Printing the calculated value

For trivial programs (e.g. less than 20 lines of code), some or all of these can be done in function *main*. However, for longer programs (or just for practice) each of these is a good candidate for an individual function.

New programmers often combine calculating a value and printing the calculated value into a single function. However, this violates the "one task" rule of thumb for functions. A function that calculates a value

should return the value to the caller and let the caller decide what to do with the calculated value (such as call another function to print the value).

02.07 — Forward declarations and definitions

Take a look at this seemingly innocent sample program:

```
#include <iostream>

int main()
{
    std::cout << "The sum of 3 and 4 is: " << add(3, 4) << '\n';
    return 0;
}

int add(int x, int y)
{
    return x + y;
}
```

You would expect this program to produce the result:

```
The sum of 3 and 4 is: 7
```

But in fact, it doesn't compile at all! Visual Studio produces the following compile error:

```
add.cpp(5) : error C3861: 'add': identifier not found
```

The reason this program doesn't compile is because the compiler compiles the contents of code files sequentially. When the compiler reaches the function call *add()* on line 5 of *main*, it doesn't know what *add* is, because we haven't defined *add* until line 9! That produces the error, *identifier not found*.

Older versions of Visual Studio would produce an additional error:

```
add.cpp(9) : error C2365: 'add'; : redefinition; previous definition
was 'formerly unknown identifier'
```

This is somewhat misleading, given that *add* wasn't ever defined in the first place. Despite this, it's useful to generally note that it is fairly common for a single error to produce many redundant or related errors or warnings. It can sometimes be hard to tell whether any error or warning beyond the first is a consequence of the first issue, or whether it is an independent issue that needs to be resolved separately.

> **Best practice**
>
> When addressing compilation errors or warnings in your programs, resolve the first issue listed and then compile again.

To fix this problem, we need to address the fact that the compiler doesn't know what add is. There are two common ways to address the issue.

Option 1: Reorder the function definitions

One way to address the issue is to reorder the function definitions so *add* is defined before *main*:

```
#include <iostream>

int add(int x, int y)
{
    return x + y;
}

int main()
{
    std::cout << "The sum of 3 and 4 is: " << add(3, 4) << '\n';
    return 0;
}
```

That way, by the time *main* calls *add*, the compiler will already know what *add* is. Because this is such a simple program, this change is relatively easy to do. However, in a larger program, it can be tedious trying to figure out which functions call which other functions (and in what order) so they can be declared sequentially.

Furthermore, this option is not always possible. Let's say we're writing a program that has two functions *A* and *B*. If function *A* calls function *B*, and function *B* calls function *A*, then there's no way to order the functions in a way that will make the compiler happy. If you define *A* first, the compiler will complain it doesn't know what *B* is. If you define *B* first, the compiler will complain that it doesn't know what *A* is.

Option 2: Use a forward declaration

We can also fix this by using a forward declaration.

A **forward declaration** allows us to tell the compiler about the existence of an identifier *before* actually defining the identifier.

In the case of functions, this allows us to tell the compiler about the existence of a function before we define the function's body. This way, when the compiler encounters a call to the function, it'll understand that we're making a function call, and can check to ensure we're calling the function correctly, even if it doesn't yet know how or where the function is defined.

To write a forward declaration for a function, we use a **function declaration** statement (also called a **function prototype**). The function declaration consists of the function's return type, name, and parameter types, terminated with a semicolon. The names of the parameters can be optionally included. The function body is not included in the declaration.

Here's a function declaration for the *add* function:

```
int add(int x, int y); // function declaration includes return type, name, parameters, and semicolon.  No function body!
```

Now, here's our original program that didn't compile, using a function declaration as a forward declaration for function *add*:

```
#include <iostream>

int add(int x, int y); // forward declaration of add() (using a function declaration)

int main()
{
    std::cout << "The sum of 3 and 4 is: " << add(3, 4) << '\n'; // this works because we forward declared add() above
    return 0;
}

int add(int x, int y) // even though the body of add() isn't defined until here
{
    return x + y;
}
```

Now when the compiler reaches the call to *add* in main, it will know what *add* looks like (a function that takes two integer parameters and returns an integer), and it won't complain.

It is worth noting that function declarations do not need to specify the names of the parameters (as they are not considered to be part of the function declaration). In the above code, you can also forward declare your function like this:

```
int add(int, int); // valid function declaration
```

However, we prefer to name our parameters (using the same names as the actual function). This allows you to understand what the function parameters are just by looking at the declaration. For example, if you were to see the declaration `void doSomething(int, int, int)`, you may think you remember what each of the parameters represent, but you may also get it wrong.

Also many automated documentation generation tools will generate documentation from the content of header files, which is where declarations are often placed. We discuss header files and declarations in lesson 2.11 – Header files.

> **Best practice**
>
> Keep the parameter names in your function declarations.

> **Tip**
>
> You can easily create function declarations by copy/pasting your function's header and adding a semicolon.

Why forward declarations?

You may be wondering why we would use a forward declaration if we could just reorder the functions to make our programs work.

Most often, forward declarations are used to tell the compiler about the existence of some function that has been defined in a different code file. Reordering isn't possible in this scenario because the caller

and the callee are in completely different files! We'll discuss this in more detail in the next lesson (2.8 – Programs with multiple code files).

Forward declarations can also be used to define our functions in an order-agnostic manner. This allows us to define functions in whatever order maximizes organization (e.g. by clustering related functions together) or reader understanding.

Less often, there are times when we have two functions that call each other. Reordering isn't possible in this case either, as there is no way to reorder the functions such that each is before the other. Forward declarations give us a way to resolve such circular dependencies.

Forgetting the function body

New programmers often wonder what happens if they forward declare a function but do not define it.

The answer is: it depends. If a forward declaration is made, but the function is never called, the program will compile and run fine. However, if a forward declaration is made and the function is called, but the program never defines the function, the program will compile okay, but the linker will complain that it can't resolve the function call.

Consider the following program:

```
#include <iostream>

int add(int x, int y); // forward declaration of add()

int main()
{
    std::cout << "The sum of 3 and 4 is: " << add(3, 4) << '\n';
    return 0;
}

// note: No definition for function add
```

In this program, we forward declare *add*, and we call *add*, but we never define *add* anywhere. When we try and compile this program, Visual Studio produces the following message:

```
Compiling...
add.cpp
Linking...
add.obj : error LNK2001: unresolved external symbol "int __cdecl
add(int,int)" (?add@@YAHHH@Z)
add.exe : fatal error LNK1120: 1 unresolved externals
```

As you can see, the program compiled okay, but it failed at the link stage because *int add(int, int)* was never defined.

Other types of forward declarations

Forward declarations are most often used with functions. However, forward declarations can also be used with other identifiers in C++, such as variables and types. Variables and types have a different syntax for forward declaration, so we'll cover these in future lessons.

Declarations vs. definitions

In C++, you'll frequently hear the words "declaration" and "definition" used, and often interchangeably. What do they mean? You now have enough fundamental knowledge to understand the difference between the two.

A **declaration** tells the *compiler* about the *existence* of an identifier and its associated type information. Here are some examples of declarations:

```
        int add(int x, int y); // tells the compiler about a function named
"add" that takes two int parameters and returns an int.  No body!
        int x;                 // tells the compiler about an integer
variable named x
```

A **definition** is a declaration that actually implements (for functions and types) or instantiates (for variables) the identifier.

Here are some examples of definitions:

```
        // because this function has a body, it is an implementation of
function add()
        int add(int x, int y)
        {
            int z{ x + y };   // instantiates variable z

            return z;
        }

        int x;                // instantiates variable x
```

In C++, all definitions are declarations. Therefore `int x;` is both a definition and a declaration.

Conversely, not all declarations are definitions. Declarations that aren't definitions are called **pure declarations**. Types of pure declarations include forward declarations for function, variables, and types.

> **Nomenclature**
>
> In common language, the term "declaration" is typically used to mean "a pure declaration", and "definition" is used to mean "a definition that also serves as a declaration". Thus, we'd typically call `int x;` a definition, even though it is both a definition and a declaration.

When the compiler encounters an identifier, it will check to ensure use of that identifier is valid (e.g. that the identifier is in scope, that it is used in a syntactically valid manner, etc…).

In most cases, a declaration is sufficient to allow the compiler to ensure an identifier is being used properly. For example, when the compiler encounters function call `add(5, 6)`, if it has already seen the declaration for `add(int, int)`, then it can validate that `add` is actually a function that takes two `int` parameters. It does not need to have actually seen the definition for function `add` (which may exist in some other file).

However, there are a few cases where the compiler must be able to see a full definition in order to use an identifier (such as for template definitions and type definitions, both of which we will discuss in future lessons).

Here's a summary table:

Term	Technical Meaning	Examples
Declaration	Tells compiler about an identifier and its associated type information.	`void foo();` // function forward declaration (no body) `void goo() {};` // function definition (has body) `int x;` // variable definition
Definition	Implements a function or instantiates a variable. Definitions are also declarations.	`void foo() { }` // function definition (has body) `int x;` // variable definition
Pure declaration	A declaration that isn't a definition.	`void foo();` // function forward declaration (no body)
Initialization	Provides an initial value for a defined object.	`int x { 2 };` // x is initialized to value 2

The term "declaration" is commonly used to mean "pure declaration", and the term "definition" used for anything that is both a definition and a declaration. We use this common nomenclature in the example column comments.

The one definition rule (ODR)

The **one definition rule** (or ODR for short) is a well-known rule in C++. The ODR has three parts:

1. Within a *file*, each function, variable, type, or template in a given scope can only have one definition. Definitions occurring in different scopes (e.g. local variables defined inside different functions, or functions defined inside different namespaces) do not violate this rule.
2. Within a *program*, each function or variable in a given scope can only have one definition. This rule exists because programs can have more than one file (we'll cover this in the next lesson). Functions and variables not visible to the linker are excluded from this rule (discussed further in lesson 7.6 – Internal linkage).
3. Types, templates, inline functions, and inline variables are allowed to have duplicate definitions in different files, so long as each definition is identical. We haven't covered what most of these things are yet, so don't worry about this for now – we'll bring it back up when it's relevant.

Violating part 1 of the ODR will cause the compiler to issue a redefinition error. Violating ODR part 2 will cause the linker to issue a redefinition error. Violating ODR part 3 will cause undefined behavior.

Here's an example of a violation of part 1:

```
int add(int x, int y)
{
     return x + y;
}

int add(int x, int y) // violation of ODR, we've already defined function add(int, int)
{
     return x + y;
}

int main()
{
    int x{};
    int x{ 5 }; // violation of ODR, we've already defined x
}
```

In this example, function `add(int, int)` is defined twice (in the global scope), and local variable `int x` is defined twice (in the scope of `main()`). The Visual Studio compiler thus issues the following compile errors:

```
project3.cpp(9): error C2084: function 'int add(int,int)' already
has a body
project3.cpp(3): note: see previous definition of 'add'
project3.cpp(16): error C2086: 'int x': redefinition
project3.cpp(15): note: see declaration of 'x'
```

However, it is not a violation of ODR part 1 for `main()` to have a local variable defined as `int x` and `add()` to also have a function parameter defined as `int x`. These definitions occur in different scopes (in the scope of each respective function), so they are considered to be separate definitions for two distinct objects, not a definition and redefinition of the same object.

> **For advanced readers**
>
> Functions that share an identifier but have different sets of parameters are also considered to be distinct functions, so such definitions do not violate the ODR. We discuss this further in lesson 11.1 – Introduction to function overloading.

02.08 — Programs with multiple code files

Adding files to your project

As programs get larger, it is common to split them into multiple files for organizational or reusability purposes. One advantage of working with an IDE is that they make working with multiple files much easier. You already know how to create and compile single-file projects. Adding new files to existing projects is very easy.

> **Best practice**

> When you add new code files to your project, give them a `.cpp` extension.

For Visual Studio users

In Visual Studio, right click on the *Source Files* folder (or the project name) in the Solution Explorer window, and choose *Add > New Item...*.

Make sure you have *C++ File (.cpp)* selected. Give the new file a name, and it will be added to your project.

Note: Your Visual Studio may opt to show you a compact view instead of the full view shown above. You can either use the compact view, or click "Show all Templates" to get to the full view.

Note: If you create a new file from the *File menu* instead of from your project in the Solution Explorer, the new file won't be added to your project automatically. You'll have to add it to the project manually. To do so, right click on *Source Files* in the *Solution Explorer*, choose *Add > Existing Item*, and then select your file.

Now when you compile your program, you should see the compiler list the name of your file as it compiles it.

For Code::Blocks users

In Code::Blocks, go to the *File menu* and choose *New > File...*.

In the *New from template* dialog, select *C/C++ source* and click *Go*.

You may or may not see a *welcome to the C/C++ source file wizard* dialog at this point. If you do, click *Next*.

On the next page of the wizard, select "C++" and click *Next*.

Now give the new file a name (don't forget the .cpp extension), and click the *All* button to ensure all build targets are selected. Finally, select *finish*.

Now when you compile your program, you should see the compiler list the name of your file as it compiles it.

For gcc users

From the command line, you can create the additional file yourself, using your favorite editor, and give it a name. When you compile your program, you'll need to include all of the relevant code files on the compile line. For example: *g++ main.cpp add.cpp -o main*, where *main.cpp* and *add.cpp* are the names of your code files, and *main* is the name of the output file.

For VS Code users

To create a new file, choose *View > Explorer* from the top nav to open the Explorer pane, and then click the *New File* icon to the right of the project name. Alternately, choose *File > New File* from the top nav. Then give your new file a name (don't forget the .cpp extension). If the file appears inside the *.vscode* folder, drag it up one level to the project folder.

Next open the *tasks.json* file, and find the line `"${file}",`.

You have two options here:

- If you wish to be explicit about what files get compiled, replace `"${file}"`, with the name of each file you wish to compile, one per line, like this:

  `"main.cpp", "add.cpp",`

- Reader "geo" reports that you can have VS Code automatically compile all .cpp files in the directory by replacing `"${file}"`, with `"${fileDirname}\\**.cpp"` (on Windows).

- Reader "Orb" reports that `"${fileDirname}/**.cpp"` works on Unix.

A multi-file example

In lesson 2.7 – Forward declarations and definitions, we took a look at a single-file program that wouldn't compile:

```
#include <iostream>

int main()
{
    std::cout << "The sum of 3 and 4 is: " << add(3, 4) << '\n';
    return 0;
}

int add(int x, int y)
{
    return x + y;
}
```

When the compiler reaches the function call to *add* on line 5 of *main*, it doesn't know what *add* is, because we haven't defined *add* until line 9! Our solution to this was to either reorder the functions (placing *add* first) or use a forward declaration for *add*.

Now let's take a look at a similar multi-file program:

add.cpp:

```
int add(int x, int y)
{
    return x + y;
}
```

main.cpp:

```
#include <iostream>

int main()
{
    std::cout << "The sum of 3 and 4 is: " << add(3, 4) << '\n'; //
compile error
    return 0;
}
```

Your compiler may compile either *add.cpp* or *main.cpp* first. Either way, *main.cpp* will fail to compile, giving the same compiler error as the previous example:

```
main.cpp(5) : error C3861: 'add': identifier not found
```

The reason is exactly the same as well: when the compiler reaches line 5 of *main.cpp*, it doesn't know what identifier *add* is.

Remember, the compiler compiles each file individually. It does not know about the contents of other code files, or remember anything it has seen from previously compiled code files. So even though the compiler may have seen the definition of function *add* previously (if it compiled *add.cpp* first), it doesn't remember.

This limited visibility and short memory is intentional, for a few reasons:

1. It allows the source files of a project to be compiled in any order.
2. When we change a source file, only that source file needs to be recompiled.
3. It reduces the possibility of naming conflicts between identifiers in different files.

We'll explore what happens when names do conflict in the next lesson (2.9 – Naming collisions and an introduction to namespaces).

Our options for a solution here are the same as before: place the definition of function *add* before function *main*, or satisfy the compiler with a forward declaration. In this case, because function *add* is in another file, the reordering option isn't possible.

The solution here is to use a forward declaration:

main.cpp (with forward declaration):

```
#include <iostream>

int add(int x, int y); // needed so main.cpp knows that add() is a function defined elsewhere

int main()
{
    std::cout << "The sum of 3 and 4 is: " << add(3, 4) << '\n';
    return 0;
}
```

add.cpp (stays the same):

```
int add(int x, int y)
{
    return x + y;
}
```

Now, when the compiler is compiling *main.cpp*, it will know what identifier *add* is and be satisfied. The linker will connect the function call to *add* in *main.cpp* to the definition of function *add* in *add.cpp*.

Using this method, we can give files access to functions that live in another file.

Try compiling *add.cpp* and the *main.cpp* with the forward declaration for yourself. If you get a linker error, make sure you've added *add.cpp* to your project or compilation line properly.

> **Tip**

> Because the compiler compiles each code file individually (and then forgets what it has seen), each code file that uses `std::cout` or `std::cin` needs to `#include <iostream>`.
>
> In the above example, if `add.cpp` had used `std::cout` or `std::cin`, it would have needed to `#include <iostream>`.

> **Key insight**
>
> When an identifier is used in an expression, the identifier must be connected to its definition.
>
> - If the compiler has seen neither a forward declaration nor a definition for the identifier in the file being compiled, it will error at the point where the identifier is used.
> - Otherwise, if a definition exists in the same file, the compiler will connect the use of the identifier to its definition.
> - Otherwise, if a definition exists in a different file (and is visible to the linker), the linker will connect the use of the identifier to its definition.
> - Otherwise, the linker will issue an error indicating that it couldn't find a definition for the identifier.

Something went wrong!

There are plenty of things that can go wrong the first time you try to work with multiple files. If you tried the above example and ran into an error, check the following:

1. If you get a compiler error about *add* not being defined in *main*, you probably forgot the forward declaration for function *add* in *main.cpp*.
2. If you get a linker error about *add* not being defined, e.g.

```
        unresolved external symbol "int __cdecl add(int,int)" (?
add@@YAHHH@Z) referenced in function _main
```

2a. …the most likely reason is that *add.cpp* is not added to your project correctly. When you compile, you should see the compiler list both *main.cpp* and *add.cpp*. If you only see *main.cpp*, then *add.cpp* definitely isn't getting compiled. If you're using Visual Studio or Code::Blocks, you should see *add.cpp* listed in the Solution Explorer/project pane on the left or right side of the IDE. If you don't, right click on your project, and add the file, then try compiling again. If you're compiling on the command line, don't forget to include both *main.cpp* and *add.cpp* in your compile command.

2b. …it's possible that you added *add.cpp* to the wrong project.

2c. …it's possible that the file is set to not compile or link. Check the file properties and ensure the file is configured to be compiled/linked. In Code::Blocks, compile and link are separate checkboxes that should be checked. In Visual Studio, there's an "exclude from build" option that should be set to "no" or left blank. Make sure you check each build configuration (e.g. debug and release) separately.

3. Do *not* `#include "add.cpp"` from *main.cpp*. While doing so compiles in this case, #including .cpp files increases the risk of naming conflicts and other unanticipated consequences (especially as programs get larger and more complex). We discuss #include further in lesson 2.10 – Introduction to the preprocessor.

Summary

C++ is designed so that each source file can be compiled independently, with no knowledge of what is in other files. Therefore, the order in which files are actually compiled should not be relevant.

We will begin working with multiple files a lot once we get into object-oriented programming, so now's as good a time as any to make sure you understand how to add and compile multiple file projects.

Reminder: Whenever you create a new code (.cpp) file, you will need to add it to your project so that it gets compiled.

Quiz time

Question #1

Split the following program into two files (main.cpp, and input.cpp). main.cpp should have the main function, and input.cpp should have the getInteger function.

Hint: Don't forget that you'll need a forward declaration in main.cpp for function `getInteger()`.

```
#include <iostream>

int getInteger()
{
    std::cout << "Enter an integer: ";
    int x{};
    std::cin >> x;
    return x;
}

int main()
{
    int x{ getInteger() };
    int y{ getInteger() };

    std::cout << x << " + " << y << " is " << x + y << '\n';
    return 0;
}
```

input.cpp:

```
#include <iostream> // we need iostream since we use it in this file

int getInteger()
{
    std::cout << "Enter an integer: ";
    int x{};
    std::cin >> x;
    return x;
}
```

main.cpp:

```
#include <iostream> // we need iostream here too since we use it in
this file as well
```

```
int getInteger(); // forward declaration for function getInteger

int main()
{
    int x{ getInteger() };
    int y{ getInteger() };

    std::cout << x << " + " << y << " is " << x + y << '\n';
    return 0;
}
```

If you get an error from the linker regarding an undefined reference to `getInteger()`, then you probably forgot to compile *input.cpp*.

02.09 — Naming collisions and an introduction to namespaces

Let's say you are driving to a friend's house for the first time, and the address given to you is 245 Front Street in Mill City. Upon reaching Mill City, you take out your map, only to discover that Mill City actually has two different Front Streets across town from each other! Which one would you go to? Unless there were some additional clue to help you decide (e.g. you remember your friend's house is near the river) you'd have to call your friend and ask for more information. Because this would be confusing and inefficient (particularly for your mail carrier), in most countries, all street names and house addresses within a city are required to be unique.

Similarly, C++ requires that all identifiers be non-ambiguous. If two identical identifiers are introduced into the same program in a way that the compiler or linker can't tell them apart, the compiler or linker will produce an error. This error is generally referred to as a **naming collision** (or **naming conflict**).

If the colliding identifiers are introduced into the same file, the result will be a compiler error. If the colliding identifiers are introduced into separate files belonging to the same program, the result will be a linker error.

An example of a naming collision

a.cpp:

```
#include <iostream>

void myFcn(int x)
{
    std::cout << x;
}
```

main.cpp:

```
#include <iostream>

void myFcn(int x)
{
```

```
    std::cout << 2 * x;
}

int main()
{
    return 0;
}
```

When the compiler compiles this program, it will compile *a.cpp* and *main.cpp* independently, and each file will compile with no problems.

However, when the linker executes, it will link all the definitions in *a.cpp* and *main.cpp* together, and discover conflicting definitions for function `myFcn()`. The linker will then abort with an error. Note that this error occurs even though `myFcn()` is never called!

Most naming collisions occur in two cases:

1. Two (or more) identically named functions (or global variables) are introduced into separate files belonging to the same program. This will result in a linker error, as shown above.
2. Two (or more) identically named functions (or global variables) are introduced into the same file. This will result in a compiler error.

As programs get larger and use more identifiers, the odds of a naming collision being introduced increases significantly. The good news is that C++ provides plenty of mechanisms for avoiding naming collisions. Local scope, which keeps local variables defined inside functions from conflicting with each other, is one such mechanism. But local scope doesn't work for function names. So how do we keep function names from conflicting with each other?

Scope regions

Back to our address analogy for a moment, having two Front Streets was only problematic because those streets existed within the same city. On the other hand, if you had to deliver mail to two addresses, one at 245 Front Street in Mill City, and another address at 245 Front Street in Jonesville, there would be no confusion about where to go. Put another way, cities provide groupings that allow us to disambiguate addresses that might otherwise conflict with each other.

A **scope region** is an area of source code where all declared identifiers are considered distinct from names declared in other scopes (much like the cities in our analogy). Two identifiers with the same name can be declared in separate scope regions without causing a naming conflict. However, within a given scope region, all identifiers must be unique, otherwise a naming collision will result.

The body of a function is one example of a scope region. Two identically-named identifiers can be defined in separate functions without issue – because each function provides a separate scope region, there is no collision. However, if you try to define two identically-named identifiers within the same function, a naming collision will result, and the compiler will complain.

Namespaces

A **namespace** provides another type of scope region (called **namespace scope**) that allows you to declare or define names inside of it for the purpose of disambiguation. The names declared in a namespace are isolated from names declared in other scopes, allowing such names to exist without conflict.

> **Key insight**
>
> A name declared within a scope region (such as a namespace) is distinct from any identical name declared in another scope.

For example, two functions with identical declarations can be defined inside different namespaces, and no naming collision or ambiguity will occur.

Namespaces may only contain declarations and definitions (e.g. variables and functions). Executable statements are not allowed unless they are part of a definition (e.g. within a function).

> **Key insight**
>
> A namespace may only contain declarations and definitions. Executable statements are only allowed as part of a definition (e.g. of a function).

Namespaces are often used to group related identifiers in a large project to help ensure they don't inadvertently collide with other identifiers. For example, if you put all your math functions in a namespace named `math`, then your math functions won't collide with identically named functions outside the `math` namespace.

We'll talk about how to create your own namespaces in a future lesson.

The global namespace

In C++, any name that is not defined inside a class, function, or a namespace is considered to be part of an implicitly-defined namespace called the **global namespace** (sometimes also called **the global scope**).

In the example at the top of the lesson, functions `main()` and both versions of `myFcn()` are defined inside the global namespace. The naming collision encountered in the example happens because both versions of `myFcn()` end up inside the global namespace, which violates the rule that all names in the scope region must be unique.

We discuss the global namespace in more detail in lesson 7.4 – Introduction to global variables.

For now, there are two things you should know:

- Identifiers declared inside the global scope are in scope from the point of declaration to the end of the file.
- Although variables can be defined in the global namespace, this should generally be avoided (we discuss why in lesson 7.8 – Why (non-const) global variables are evil).

For example:

```
#include <iostream> // imports the declaration of std::cout into the
```

```
global scope

        // All of the following statements are part of the global namespace

        void foo();    // okay: function forward declaration
        int x;         // compiles but strongly discouraged: non-const
global variable definition (without initializer)
        int y { 5 };   // compiles but strongly discouraged: non-const
global variable definition (with initializer)
        x = 5;         // compile error: executable statements are not
allowed in namespaces

        int main()     // okay: function definition
        {
            return 0;
        }

        void goo();    // okay: A function forward declaration
```

The std namespace

When C++ was originally designed, all of the identifiers in the C++ standard library (including `std::cin` and `std::cout`) were available to be used without the `std::` prefix (they were part of the global namespace). However, this meant that any identifier in the standard library could potentially conflict with any name you picked for your own identifiers (also defined in the global namespace). Code that was once working might suddenly have a naming conflict when you include a different part of the standard library. Or worse, code that compiled under one version of C++ might not compile under the next version of C++, as new identifiers introduced into the standard library could have a naming conflict with already written code. So C++ moved all of the functionality in the standard library into a namespace named `std` (short for "standard").

It turns out that `std::cout`'s name isn't really `std::cout`. It's actually just `cout`, and `std` is the name of the namespace that identifier `cout` is part of. Because `cout` is defined in the `std` namespace, the name `cout` won't conflict with any objects or functions named `cout` that we create outside of the `std` namespace (such as in the global namespace).

> **Key insight**
>
> When you use an identifier that is defined inside a non-global namespace (e.g. the `std` namespace), you need to tell the compiler that the identifier lives inside the namespace.

There are a few different ways to do this.

Explicit namespace qualifier std::

The most straightforward way to tell the compiler that we want to use `cout` from the `std` namespace is by explicitly using the `std::` prefix. For example:

```
        #include <iostream>

        int main()
```

```
{
    std::cout << "Hello world!"; // when we say cout, we mean the cout defined in the std namespace
    return 0;
}
```

The `::` symbol is an operator called the **scope resolution operator**. The identifier to the left of the `::` symbol identifies the namespace that the name to the right of the `::` symbol is contained within. If no identifier to the left of the `::` symbol is provided, the global namespace is assumed.

So when we say `std::cout` we're saying "the `cout` that is declared in namespace `std`".

This is the safest way to use `cout`, because there's no ambiguity about which `cout` we're referencing (the one in the `std` namespace).

> **Best practice**
>
> Use explicit namespace prefixes to access identifiers defined in a namespace.

When an identifier includes a namespace prefix, the identifier is called a **qualified name**.

Using namespace std (and why to avoid it)

Another way to access identifiers inside a namespace is to use a using-directive statement. Here's our original "Hello world" program with a using-directive:

```
#include <iostream>

using namespace std; // this is a using-directive that allows us to access names in the std namespace with no namespace prefix

int main()
{
    cout << "Hello world!";
    return 0;
}
```

A **using directive** allows us to access the names in a namespace without using a namespace prefix. So in the above example, when the compiler goes to determine what identifier `cout` is, it will match with `std::cout`, which, because of the using-directive, is accessible as just `cout`.

Many texts, tutorials, and even some IDEs recommend or use a using-directive at the top of the program. However, used in this way, this is a bad practice, and highly discouraged.

Consider the following program:

```
#include <iostream> // imports the declaration of std::cout into the global scope

using namespace std; // makes std::cout accessible as "cout"

int cout() // defines our own "cout" function in the global
```

```
namespace
        {
            return 5;
        }

        int main()
        {
            cout << "Hello, world!"; // Compile error!  Which cout do we
want here?  The one in the std namespace or the one we defined
above?

            return 0;
        }
```

The above program doesn't compile, because the compiler now can't tell whether we want the `cout` function that we defined, or `std::cout`.

When using a using-directive in this manner, *any* identifier we define may conflict with *any* identically named identifier in the `std` namespace. Even worse, while an identifier name may not conflict today, it may conflict with new identifiers added to the std namespace in future language revisions. This was the whole point of moving all of the identifiers in the standard library into the `std` namespace in the first place!

> **Warning**
>
> Avoid using-directives (such as `using namespace std;`) at the top of your program or in header files. They violate the reason why namespaces were added in the first place.

> **Related content**
>
> We talk more about using-declarations and using-directives (and how to use them responsibly) in lesson 7.13 – Using declarations and using directives.

Curly braces and indented code

In C++, curly braces are often used to delineate a scope region that is nested within another scope region (braces are also used for some non-scope-related purposes, such as list initialization). For example, a function defined inside the global scope region uses curly braces to separate the scope region of the function from the global scope.

In certain cases, identifiers defined outside the curly braces may still be part of the scope defined by the curly braces rather than the surrounding scope – function parameters are a good example of this.

For example:

```
        #include <iostream> // imports the declaration of std::cout into the
global scope

        void foo(int x) // foo is defined in the global scope, x is defined
within scope of foo()
        { // braces used to delineate nested scope region for function foo()
            std::cout << x << '\n';
        } // x goes out of scope here
```

```
int main()
{ // braces used to delineate nested scope region for function main()
    foo(5);

    int x { 6 }; // x is defined within the scope of main()
    std::cout << x << '\n';

    return 0;
} // x goes out of scope here
// foo and main (and std::cout) go out of scope here (the end of the file)
```

The code that exists inside a nested scope region is conventionally indented one level, both for readability and to help indicate that it exists inside a separate scope region.

The `#include` and function definitions for `foo()` and `main()` exist in the global scope region, so they are not indented. The statements inside each function exist inside the nested scope region of the function, so they are indented one level.

02.10 — Introduction to the preprocessor

When you compile your project, you might expect that the compiler compiles each code file exactly as you've written it. This actually isn't the case.

Instead, prior to compilation, each code (.cpp) file goes through a **preprocessing** phase. In this phase, a program called the **preprocessor** makes various changes to the text of the code file. The preprocessor does not actually modify the original code files in any way – rather, all changes made by the preprocessor happen either temporarily in-memory or using temporary files.

> **As an aside…**
>
> Historically, the preprocessor was a separate program from the compiler, but in modern compilers, the preprocessor may be built right into the compiler itself.

Most of what the preprocessor does is fairly uninteresting. For example, it strips out comments, and ensures each code file ends in a newline. However, the preprocessor does have one very important role: it is what processes `#include` directives (which we'll discuss more in a moment).

When the preprocessor has finished processing a code file, the result is called a **translation unit**. This translation unit is what is then compiled by the compiler.

> **Related content**
>
> The entire process of preprocessing, compiling, and linking is called **translation**.

> If you're curious, here is a list of translation phases. As of the time of writing, preprocessing encompasses phases 1 through 4, and compilation is phases 5 through 7.

Preprocessor directives

When the preprocessor runs, it scans through the code file (from top to bottom), looking for preprocessor directives. **Preprocessor directives** (often just called *directives*) are instructions that start with a # symbol and end with a newline (NOT a semicolon). These directives tell the preprocessor to perform certain text manipulation tasks. Note that the preprocessor does not understand C++ syntax – instead, the directives have their own syntax (which in some cases resembles C++ syntax, and in other cases, not so much).

> **Key insight**
>
> The final output of the preprocessor contains no directives – only the output of the processed directive is passed to the compiler.

> **As an aside…**
>
> `Using directives` (introduced in lesson 2.9 – Naming collisions and an introduction to namespaces) are not preprocessor directives (and thus are not processed by the preprocessor). So while the term `directive` *usually* means a `preprocessor directive`, this is not always the case.

#Include

You've already seen the `#include` directive in action (generally to `#include <iostream>`). When you `#include` a file, the preprocessor replaces the `#include` directive with the contents of the included file. The included contents are then preprocessed (which may result in additional `#includes` being preprocessed recursively), then the rest of the file is preprocessed.

Consider the following program:

```
#include <iostream>

int main()
{
    std::cout << "Hello, world!\n";
    return 0;
}
```

When the preprocessor runs on this program, the preprocessor will replace `#include <iostream>` with the contents of the file named "iostream" and then preprocess the included content and the rest of the file.

Since `#include` is almost exclusively used to include header files, we'll discuss `#include` in more detail in the next lesson (when we discuss header files).

> **Key insight**

> Each translation unit typically consists of a single code (.cpp) file and all header files it `#includes` (applied recursively, since header files can `#include` other header files).

Macro defines

The `#define` directive can be used to create a macro. In C++, a **macro** is a rule that defines how input text is converted into replacement output text.

There are two basic types of macros: *object-like macros*, and *function-like macros*.

Function-like macros act like functions, and serve a similar purpose. Their use is generally considered unsafe, and almost anything they can do can be done by a normal function.

Object-like macros can be defined in one of two ways:

```
#define IDENTIFIER
#define IDENTIFIER substitution_text
```

The top definition has no substitution text, whereas the bottom one does. Because these are preprocessor directives (not statements), note that neither form ends with a semicolon.

The identifier for a macro uses the same naming rules as normal identifiers: they can use letters, numbers, and underscores, cannot start with a number, and should not start with an underscore. By convention, macro names are typically all uppercase, separated by underscores.

> **Best practice**
>
> Macro names should be written in all uppercase letters, with words separated by underscores.

Object-like macros with substitution text

When the preprocessor encounters this directive, an association is made between the macro identifier and *substitution_text*. All further occurrences of the macro identifier (outside of use in other preprocessor commands) are replaced by the *substitution_text*.

Consider the following program:

```
#include <iostream>

#define MY_NAME "Alex"

int main()
{
    std::cout << "My name is: " << MY_NAME << '\n';

    return 0;
}
```

The preprocessor converts the above into the following:

```
// The contents of iostream are inserted here
```

```
int main()
{
    std::cout << "My name is: " << "Alex" << '\n';

    return 0;
}
```

Which, when run, prints the output `My name is: Alex`.

Object-like macros with substitution text were used (in C) as a way to assign names to literals. This is no longer necessary, as better methods are available in C++ (see 7.10 – Sharing global constants across multiple files (using inline variables)). Object-like macros with substitution text are now mostly seen in legacy code, and we recommend avoiding them whenever possible.

> **Best practice**
>
> Avoid macros with substitution text unless no viable alternatives exist.

Object-like macros without substitution text

Object-like macros can also be defined without substitution text.

For example:

```
#define USE_YEN
```

Macros of this form work like you might expect: most further occurrences of the identifier is removed and replaced by nothing!

This might seem pretty useless, and it *is useless* for doing text substitution. However, that's not what this form of the directive is generally used for. We'll discuss the uses of this form in just a moment.

Unlike object-like macros with substitution text, macros of this form are generally considered acceptable to use.

Conditional compilation

The *conditional compilation* preprocessor directives allow you to specify under what conditions something will or won't compile. There are quite a few different conditional compilation directives, but we'll only cover a few that are used the most often: `#ifdef`, `#ifndef`, and `#endif`.

The `#ifdef` preprocessor directive allows the preprocessor to check whether an identifier has been previously defined via `#define`. If so, the code between the `#ifdef` and matching `#endif` is compiled. If not, the code is ignored.

Consider the following program:

```
#include <iostream>

#define PRINT_JOE
```

```
int main()
{
#ifdef PRINT_JOE
    std::cout << "Joe\n"; // will be compiled since PRINT_JOE is defined
#endif

#ifdef PRINT_BOB
    std::cout << "Bob\n"; // will be excluded since PRINT_BOB is not defined
#endif

    return 0;
}
```

Because PRINT_JOE has been `#defined`, the line `std::cout << "Joe\n"` will be compiled. Because PRINT_BOB has not been `#defined`, the line `std::cout << "Bob\n"` will be ignored.

`#ifndef` is the opposite of `#ifdef`, in that it allows you to check whether an identifier has *NOT* been `#defined` yet.

```
#include <iostream>

int main()
{
#ifndef PRINT_BOB
    std::cout << "Bob\n";
#endif

    return 0;
}
```

This program prints "Bob", because PRINT_BOB was never `#defined`.

In place of `#ifdef PRINT_BOB` and `#ifndef PRINT_BOB`, you'll also see `#if defined(PRINT_BOB)` and `#if !defined(PRINT_BOB)`. These do the same, but use a slightly more C++-style syntax.

You can see a practical use of this feature in lesson 0.13 – What language standard is my compiler using?.

#if 0

One more common use of conditional compilation involves using `#if 0` to exclude a block of code from being compiled (as if it were inside a comment block):

```
#include <iostream>

int main()
{
    std::cout << "Joe\n";

#if 0 // Don't compile anything starting here
    std::cout << "Bob\n";
    std::cout << "Steve\n";
#endif // until this point

    return 0;
```

```
}
```

The above code only prints “Joe”, because “Bob” and “Steve” are excluded from compilation by the `#if 0` preprocessor directive.

This provides a convenient way to “comment out” code that contains multi-line comments (which can’t be commented out using another multi-line comment due to multi-line comments being non-nestable):

```
#include <iostream>

int main()
{
    std::cout << "Joe\n";

#if 0 // Don't compile anything starting here
    std::cout << "Bob\n";
    /* Some
     * multi-line
     * comment here
     */
    std::cout << "Steve\n";
#endif // until this point

    return 0;
}
```

To temporarily re-enable code that has been wrapped in an `#if 0`, you can change the `#if 0` to `#if 1`:

```
#include <iostream>

int main()
{
    std::cout << "Joe\n";

#if 1 // always true, so the following code will be compiled
    std::cout << "Bob\n";
    /* Some
     * multi-line
     * comment here
     */
    std::cout << "Steve\n";
#endif

    return 0;
}
```

Macro substitution within other preprocessor commands

Now you might be wondering, given the following code:

```
#define PRINT_JOE

int main()
{
#ifdef PRINT_JOE
    std::cout << "Joe\n"; // will be compiled since PRINT_JOE is
```

```
defined
        #endif

            return 0;
        }
```

Since we defined `PRINT_JOE` to be nothing, how come the preprocessor didn't replace `PRINT_JOE` in `#ifdef PRINT_JOE` with nothing and exclude the output statement from compilation?

In most cases, macro substitution does not occur when a macro identifier is used within another preprocessor command.

> **As an aside…**
>
> There is at least one exception to this rule: most forms of `#if` and `#elif` do macro substitution within the preprocessor command.

As another example:

```
        #define FOO 9 // Here's a macro substitution

        #ifdef FOO // This FOO does not get replaced with 9 because it's
part of another preprocessor directive
            std::cout << FOO << '\n'; // This FOO gets replaced with 9
because it's part of the normal code
        #endif
```

The scope of #defines

Directives are resolved before compilation, from top to bottom on a file-by-file basis.

Consider the following program:

```
        #include <iostream>

        void foo()
        {
        #define MY_NAME "Alex"
        }

        int main()
        {
            std::cout << "My name is: " << MY_NAME << '\n';

            return 0;
        }
```

Even though it looks like `#define MY_NAME "Alex"` is defined inside function `foo`, the preprocessor doesn't understand C++ concepts like functions. Therefore, this program behaves identically to one where `#define MY_NAME "Alex"` was defined either before or immediately after function `foo`. To avoid confusion, you'll generally want to `#define` identifiers outside of functions.

Because an `#include` directive replaces the `#include` directive with the content of the included file, an `#include` can copy directives from the included file into the current file. These directives will then be processed in order.

For example, the following also behaves identically to the prior examples:

Alex.h:

```
#define MY_NAME "Alex"
```

main.cpp:

```
#include "Alex.h" // copies #define MY_NAME from Alex.h here
#include <iostream>

int main()
{
    std::cout << "My name is: " << MY_NAME << '\n'; // preprocessor
replaces MY_NAME with "Alex"

    return 0;
}
```

Once the preprocessor has finished, all defined identifiers from that file are discarded. This means that directives are only valid from the point of definition to the end of the file in which they are defined. Directives defined in one file do not have any impact on other files (unless they are `#included` into another file). For example:

function.cpp:

```
#include <iostream>

void doSomething()
{
#ifdef PRINT
    std::cout << "Printing!\n";
#endif
#ifndef PRINT
    std::cout << "Not printing!\n";
#endif
}
```

main.cpp:

```
void doSomething(); // forward declaration for function
doSomething()

#define PRINT

int main()
{
    doSomething();

    return 0;
}
```

The above program will print:

```
Not printing!
```

Even though PRINT was defined in `main.cpp`, that doesn't have any impact on any of the code in `function.cpp` (PRINT is only `#defined` from the point of definition to the end of main.cpp). This will be of consequence when we discuss header guards in a future lesson.

02.11 — Header files

In lesson 2.8 – Programs with multiple code files, we discussed how programs can be split across multiple files. We also discussed how forward declarations are used to allow the code in one file to access something defined in another file.

When programs contain only a few small files, manually adding a few forward declarations to the top of each file isn't too bad. However, as programs grow larger (and make use of more files and functions), having to manually add a large number of (possibly different) forward declarations to the top of each file becomes extremely tedious. For example, if you have a 5 file program, each of which requires 10 forward declarations, you're going to have to copy/paste in 50 forward declarations. Now consider the case where you have 100 files and they each require 100 forward declarations. This simply doesn't scale!

To address this issue, C++ programs typically take a different approach.

Header files

C++ code files (with a .cpp extension) are not the only files commonly seen in C++ programs. The other type of file is called a **header file**. Header files usually have a .h extension, but you will occasionally see them with a .hpp extension or no extension at all.

Conventionally, header files are used to propagate a bunch of related forward declarations into a code file.

> **Key insight**
>
> Header files allow us to put declarations in one place and then import them wherever we need them. This can save a lot of typing in multi-file programs.

Using standard library header files

Consider the following program:

```
#include <iostream>

int main()
{
    std::cout << "Hello, world!";
    return 0;
}
```

This program prints "Hello, world!" to the console using *std::cout*. However, this program never provided a definition or declaration for *std::cout*, so how does the compiler know what *std::cout* is?

The answer is that *std::cout* has been forward declared in the "iostream" header file. When we `#include <iostream>`, we're requesting that the preprocessor copy all of the content (including

forward declarations for std::cout) from the file named “iostream” into the file doing the #include.

> **Key insight**
>
> When you `#include` a file, the content of the included file is inserted at the point of inclusion. This provides a useful way to pull in declarations from another file.

Consider what would happen if the *iostream* header did not exist. Wherever you used *std::cout*, you would have to manually type or copy in all of the declarations related to *std::cout* into the top of each file that used *std::cout*! This would require a lot of knowledge about how *std::cout* was declared, and would be a ton of work. Even worse, if a function prototype was added or changed, we’d have to go manually update all of the forward declarations.

It’s much easier to just `#include <iostream>`!

Using header files to propagate forward declarations

Now let’s go back to the example we were discussing in a previous lesson. When we left off, we had two files, *add.cpp* and *main.cpp*, that looked like this:

add.cpp:

```
int add(int x, int y)
{
    return x + y;
}
```

main.cpp:

```
#include <iostream>

int add(int x, int y); // forward declaration using function prototype

int main()
{
    std::cout << "The sum of 3 and 4 is " << add(3, 4) << '\n';
    return 0;
}
```

(If you’re recreating this example from scratch, don’t forget to add *add.cpp* to your project so it gets compiled in).

In this example, we used a forward declaration so that the compiler will know what identifier *add* is when compiling *main.cpp*. As previously mentioned, manually adding forward declarations for every function you want to use that lives in another file can get tedious quickly.

Let’s write a header file to relieve us of this burden. Writing a header file is surprisingly easy, as header files only consist of two parts:

1. A *header guard*, which we’ll discuss in more detail in the next lesson (2.12 – Header guards).

2. The actual content of the header file, which should be the forward declarations for all of the identifiers we want other files to be able to see.

Adding a header file to a project works analogously to adding a source file (covered in lesson 2.8 – Programs with multiple code files).

If using an IDE, go through the same steps and choose “Header” instead of “Source” when asked. The header file should appear as part of your project.

If using the command line, just create a new file in your favorite editor in the same directory as your source (.cpp) files. Unlike source files, header files should *not* be added to your compile command (they are implicitly included by #include statements and compiled as part of your source files).

> **Best practice**
>
> Prefer a .h suffix when naming your header files (unless your project already follows some other convention).
>
> This is a longstanding convention for C++ header files, and most IDEs still default to .h over other options.

Header files are often paired with code files, with the header file providing forward declarations for the corresponding code file. Since our header file will contain a forward declaration for functions defined in *add.cpp*, we’ll call our new header file *add.h*.

> **Best practice**
>
> If a header file is paired with a code file (e.g. add.h with add.cpp), they should both have the same base name (add).

Here’s our completed header file:

add.h:

```
        // We really should have a header guard here, but will omit it for
simplicity (we'll cover header guards in the next lesson)

        // This is the content of the .h file, which is where the
declarations go
        int add(int x, int y); // function prototype for add.h -- don't
forget the semicolon!
```

In order to use this header file in main.cpp, we have to #include it (using quotes, not angle brackets).

main.cpp:

```
        #include "add.h" // Insert contents of add.h at this point.  Note
use of double quotes here.
        #include <iostream>

        int main()
        {
            std::cout << "The sum of 3 and 4 is " << add(3, 4) << '\n';
            return 0;
        }
```

add.cpp:

```
#include "add.h" // Insert contents of add.h at this point.  Note use of double quotes here.

int add(int x, int y)
{
    return x + y;
}
```

When the preprocessor processes the `#include "add.h"` line, it copies the contents of add.h into the current file at that point. Because our *add.h* contains a forward declaration for function *add()*, that forward declaration will be copied into *main.cpp*. The end result is a program that is functionally the same as the one where we manually added the forward declaration at the top of *main.cpp*.

Consequently, our program will compile and link correctly.

Note: In the graphic above, "Standard Runtime Library" should be labelled as the "C++ Standard Library".

How including definitions in a header file results in a violation of the one-definition rule

For now, you should avoid putting function or variable definitions in header files. Doing so will generally result in a violation of the one-definition rule (ODR) in cases where the header file is included into more than one source file.

> **Related content**
>
> We covered the one-definition rule (ODR) in lesson 2.7 – Forward declarations and definitions.

Let's illustrate how this happens:

add.h:

```
// We really should have a header guard here, but will omit it for simplicity (we'll cover header guards in the next lesson)

// definition for add() in header file -- don't do this!
int add(int x, int y)
{
    return x + y;
}
```

main.cpp:

```
#include "add.h" // Contents of add.h copied here
#include <iostream>

int main()
{
    std::cout << "The sum of 3 and 4 is " << add(3, 4) << '\n';

    return 0;
}
```

add.cpp:

```
#include "add.h" // Contents of add.h copied here
```

When `main.cpp` is compiled, the `#include "add.h"` will be replaced with the contents of `add.h` and then compiled. Therefore, the compiler will compile something that looks like this:

main.cpp (after preprocessing):

```
// from add.h:
int add(int x, int y)
{
    return x + y;
}

// contents of iostream header here

int main()
{
    std::cout << "The sum of 3 and 4 is " << add(3, 4) << '\n';

    return 0;
}
```

This will compile just fine.

When the compiler compiles `add.cpp`, the `#include "add.h"` will be replaced with the contents of `add.h` and then compiled. Therefore, the compiler will compile something like this:

add.cpp (after preprocessing):

```
int add(int x, int y)
{
    return x + y;
}
```

This will also compile just fine.

Finally, the linker will run. And the linker will see that there are now two definitions for function `add()`: one in main.cpp, and one in add.cpp. This is a violation of ODR part 2, which states, “Within a given program, a variable or normal function can only have one definition.”

> **Best practice**
>
> Do not put function and variable definitions in your header files (for now).
>
> Defining either of these in a header file will likely result in a violation of the one-definition rule (ODR) if that header is then #included into more than one source (.cpp) file.

> **Author’s note**
>
> In future lessons, we will encounter additional kinds of definitions that can be safely defined in header files (because they are exempt from the ODR). This includes definitions for inline functions, inline variables, types, and templates. We’ll discuss this further when we introduce each of these.

Source files should include their paired header

In C++, it is a best practice for code files to #include their paired header file (if one exists). This allows the compiler to catch certain kinds of errors at compile time instead of link time. For example:

add.h:

```
        // We really should have a header guard here, but will omit it for
simplicity (we'll cover header guards in the next lesson)

        int add(int x, int y);
```

main.cpp:

```
        #include "add.h"
        #include <iostream>

        int main()
        {
            std::cout << "The sum of 3 and 4 is " << add(3, 4) << '\n';
            return 0;
        }
```

add.cpp:

```
        #include "add.h"         // copies forward declaration from add.h
here

        double add(int x, int y) // oops, return type is double instead of
int
        {
            return x + y;
        }
```

When *add.cpp* is compiled, the forward declaration `int add(int x, int y)` will be copied into *add.cpp* at the point of the #include. When the compiler reaches the definition `double add(int x, int y)`, it will note that the return types of the forward declaration and the definition don't match. Because functions can't differ by only a return type, the compiler will error and abort compilation immediately. In a larger project, this can save a lot of time and help pinpoint where the issue is.

> **As an aside…**
>
> Unfortunately, this doesn't work if it is a parameter with a different type instead of a return type. This is because C++ supports overloaded functions (functions with the same name but different parameter types), so the compiler will assume a function with a mismatched parameter type is a different overload. Can't win em all.

If the `#include "add.h"` is not present, the compiler won't catch the issue because it doesn't see the mismatch. We have to wait until link time for the issue to surface.

We will also see many examples in future lessons where content required by the source file is defined in the paired header. In such cases, including the header is a necessity.

> **Best practice**
>
> Source files should #include their paired header file (if one exists).

Do not #include .cpp files

Although the preprocessor will happily do so, you should generally not `#include` .cpp files. These should be added to your project and compiled.

There are number of reasons for this:

- Doing so can cause naming collisions between source files.
- In a large project it can be hard to avoid one definition rules (ODR) issues.
- Any change to such a .cpp file will cause both the .cpp file and any other .cpp file that includes it to recompile, which can take a long time. Headers tend to change less often than source files.
- It is non-conventional to do so.

> **Best practice**
>
> Avoid #including .cpp files.

> **Tip**
>
> If your project doesn't compile unless you #include .cpp files, that means those .cpp files are not being compiled as part of your project. Add them to your project or command line so they get compiled.

Troubleshooting

If you get a compiler error indicating that *add.h* isn't found, make sure the file is really named *add.h*. Depending on how you created and named it, it's possible the file could have been named something like *add* (no extension) or *add.h.txt* or *add.hpp*. Also make sure it's sitting in the same directory as the rest of your code files.

If you get a linker error about function *add* not being defined, make sure you've included *add.cpp* in your project so the definition for function *add* can be linked into the program.

Angled brackets vs double quotes

You're probably curious why we use angled brackets for `iostream`, and double quotes for `add.h`. It's possible that a header file with the same filename might exist in multiple directories. Our use of angled brackets vs double quotes helps give the preprocessor a clue as to where it should look for header files.

When we use angled brackets, we're telling the preprocessor that this is a header file we didn't write ourselves. The preprocessor will search for the header only in the directories specified by the `include directories`. The `include directories` are configured as part of your project/IDE settings/compiler settings, and typically default to the

directories containing the header files that come with your compiler and/or OS. The preprocessor will not search for the header file in your project's source code directory.

When we use double-quotes, we're telling the preprocessor that this is a header file that we wrote. The preprocessor will first search for the header file in the current directory. If it can't find a matching header there, it will then search the `include directories`.

> **Rule**
>
> Use double quotes to include header files that you've written or are expected to be found in the current directory. Use angled brackets to include headers that come with your compiler, OS, or third-party libraries you've installed elsewhere on your system.

Why doesn't iostream have a .h extension?

Another commonly asked question is "why doesn't iostream (or any of the other standard library header files) have a .h extension?". The answer is that *iostream.h* is a different header file than *iostream*! To explain requires a short history lesson.

When C++ was first created, all of the headers in the standard library ended in a *.h* suffix. These headers included:

Header type	Naming convention	Example	Identifiers placed in namespace
C++ specific	<xxx.h>	iostream.h	Global namespace
C compatibility	<xxx.h>	stddef.h	Global namespace

The original versions of *cout* and *cin* were declared in *iostream.h* in the global namespace. Life was consistent, and it was good.

When the language was standardized by the ANSI committee, they decided to move all of the names used in the standard library into the *std* namespace to help avoid naming conflicts with user-declared identifiers. However, this presented a problem: if they moved all the names into the *std* namespace, none of the old programs (that included iostream.h) would work anymore!

To work around this issue, C++ introduced new header files that lack the *.h* extension. These new header files declare all names inside the *std* namespace. This way, older programs that include `#include <iostream.h>` do not need to be rewritten, and newer programs can `#include <iostream>`.

Modern C++ now contains 4 sets of header files:

Header type	Naming convention	Example	Identifiers placed in namespace
C++ specific (new)	<xxx>	iostream	`std` namespace
			`std` namespace

C compatibility (new)	<cxxx>	cstddef	(required) global namespace (optional)
C++ specific (old)	<xxx.h>	iostream.h	Global namespace

02.12 — Header guards

The duplicate definition problem

In lesson 2.7 – Forward declarations and definitions, we noted that a variable or function identifier can only have one definition (the one definition rule). Thus, a program that defines a variable identifier more than once will cause a compile error:

```
int main()
{
    int x; // this is a definition for variable x
    int x; // compile error: duplicate definition

    return 0;
}
```

Similarly, programs that define a function more than once will also cause a compile error:

```
#include <iostream>

int foo() // this is a definition for function foo
{
    return 5;
}

int foo() // compile error: duplicate definition
{
    return 5;
}

int main()
{
    std::cout << foo();
    return 0;
}
```

While these programs are easy to fix (remove the duplicate definition), with header files, it's quite easy to end up in a situation where a definition in a header file gets included more than once. This can happen when a header file #includes another header file (which is common).

> **Author's note**
>
> In upcoming examples, we'll define some functions inside header files. You generally shouldn't do this.

> We are doing so here because it is the most effective way to demonstrate some concepts using functionality we've already covered.

Consider the following academic example:

square.h:

```
int getSquareSides()
{
    return 4;
}
```

wave.h:

```
#include "square.h"
```

main.cpp:

```
#include "square.h"
#include "wave.h"

int main()
{
    return 0;
}
```

This seemingly innocent looking program won't compile! Here's what's happening. First, *main.cpp* #includes *square.h*, which copies the definition for function *getSquareSides* into *main.cpp*. Then *main.cpp* #includes *wave.h*, which #includes *square.h* itself. This copies contents of *square.h* (including the definition for function *getSquareSides*) into *wave.h*, which then gets copied into *main.cpp*.

Thus, after resolving all of the #includes, *main.cpp* ends up looking like this:

```
int getSquareSides()  // from square.h
{
    return 4;
}

int getSquareSides() // from wave.h (via square.h)
{
    return 4;
}

int main()
{
    return 0;
}
```

Duplicate definitions and a compile error. Each file, individually, is fine. However, because *main.cpp* ends up #including the content of *square.h* twice, we've run into problems. If *wave.h* needs *getSquareSides()*, and *main.cpp* needs both *wave.h* and *square.h*, how would you resolve this issue?

Header guards

The good news is that we can avoid the above problem via a mechanism called a **header guard** (also called an **include guard**). Header guards are conditional compilation directives that take the following form:

```
#ifndef SOME_UNIQUE_NAME_HERE
#define SOME_UNIQUE_NAME_HERE

// your declarations (and certain types of definitions) here

#endif
```

When this header is #included, the preprocessor will check whether *SOME_UNIQUE_NAME_HERE* has been previously defined in this translation unit. If this is the first time we're including the header, *SOME_UNIQUE_NAME_HERE* will not have been defined. Consequently, it #defines *SOME_UNIQUE_NAME_HERE* and includes the contents of the file. If the header is included again into the same file, *SOME_UNIQUE_NAME_HERE* will already have been defined from the first time the contents of the header were included, and the contents of the header will be ignored (thanks to the #ifndef).

All of your header files should have header guards on them. *SOME_UNIQUE_NAME_HERE* can be any name you want, but by convention is set to the full filename of the header file, typed in all caps, using underscores for spaces or punctuation. For example, *square.h* would have the header guard:

square.h:

```
#ifndef SQUARE_H
#define SQUARE_H

int getSquareSides()
{
    return 4;
}

#endif
```

Even the standard library headers use header guards. If you were to take a look at the iostream header file from Visual Studio, you would see:

```
#ifndef _IOSTREAM_
#define _IOSTREAM_

// content here

#endif
```

> **For advanced readers**
>
> In large programs, it's possible to have two separate header files (included from different directories) that end up having the same filename (e.g. directoryA.h and directoryB.h). If only the filename is used for the include guard (e.g. CONFIG_H), these two files may end up using the same guard name. If that happens, any file that includes (directly or indirectly) both config.h files will not receive the contents of the include file to be included second. This will probably cause a compilation error.

> Because of this possibility for guard name conflicts, many developers recommend using a more complex/unique name in your header guards. Some good suggestions are a naming convention of PROJECT_PATH_FILE_H, FILE_LARGE-RANDOM-NUMBER_H, or FILE_CREATION-DATE_H.

Updating our previous example with header guards

Let's return to the *square.h* example, using the *square.h* with header guards. For good form, we'll also add header guards to *wave.h*.

square.h

```
#ifndef SQUARE_H
#define SQUARE_H

int getSquareSides()
{
    return 4;
}

#endif
```

wave.h:

```
#ifndef WAVE_H
#define WAVE_H

#include "square.h"

#endif
```

main.cpp:

```
#include "square.h"
#include "wave.h"

int main()
{
    return 0;
}
```

After the preprocessor resolves all of the #include directives, this program looks like this:

main.cpp:

```
// Square.h included from main.cpp
#ifndef SQUARE_H // square.h included from main.cpp
#define SQUARE_H // SQUARE_H gets defined here

// and all this content gets included
int getSquareSides()
{
    return 4;
}

#endif // SQUARE_H
```

```
#ifndef WAVE_H // wave.h included from main.cpp
#define WAVE_H
#ifndef SQUARE_H // square.h included from wave.h, SQUARE_H is
already defined from above
#define SQUARE_H // so none of this content gets included

int getSquareSides()
{
    return 4;
}

#endif // SQUARE_H
#endif // WAVE_H

int main()
{
    return 0;
}
```

Let's look at how this evaluates.

First, the preprocessor evaluates `#ifndef SQUARE_H`. `SQUARE_H` has not been defined yet, so the code from the `#ifndef` to the subsequent `#endif` is included for compilation. This code defines `SQUARE_H`, and has the definition for the `getSquareSides` function.

Later, the next `#ifndef SQUARE_H` is evaluated. This time, `SQUARE_H` is defined (because it got defined above), so the code from the `#ifndef` to the subsequent `#endif` is excluded from compilation.

Header guards prevent duplicate inclusions because the first time a guard is encountered, the guard macro isn't defined, so the guarded content is included. Past that point, the guard macro is defined, so any subsequent copies of the guarded content are excluded.

Header guards do not prevent a header from being included once into different code files

Note that the goal of header guards is to prevent a code file from receiving more than one copy of a guarded header. By design, header guards do *not* prevent a given header file from being included (once) into separate code files. This can also cause unexpected problems. Consider:

square.h:

```
#ifndef SQUARE_H
#define SQUARE_H

int getSquareSides()
{
    return 4;
}

int getSquarePerimeter(int sideLength); // forward declaration for
getSquarePerimeter

#endif
```

square.cpp:

```
#include "square.h"  // square.h is included once here

int getSquarePerimeter(int sideLength)
{
    return sideLength * getSquareSides();
}
```

main.cpp:

```
#include "square.h" // square.h is also included once here
#include <iostream>

int main()
{
    std::cout << "a square has " << getSquareSides() << " sides\n";
    std::cout << "a square of length 5 has perimeter length " <<
getSquarePerimeter(5) << '\n';

    return 0;
}
```

Note that *square.h* is included from both *main.cpp* and *square.cpp*. This means the contents of *square.h* will be included once into *square.cpp* and once into *main.cpp*.

Let's examine why this happens in more detail. When *square.h* is included from *square.cpp*, *SQUARE_H* is defined until the end of *square.cpp*. This define prevents *square.h* from being included into *square.cpp* a second time (which is the point of header guards). However, once *square.cpp* is finished, *SQUARE_H* is no longer considered defined. This means that when the preprocessor runs on *main.cpp*, *SQUARE_H* is not initially defined in *main.cpp*.

The end result is that both *square.cpp* and *main.cpp* get a copy of the definition of *getSquareSides*. This program will compile, but the linker will complain about your program having multiple definitions for identifier *getSquareSides*!

The best way to work around this issue is simply to put the function definition in one of the .cpp files so that the header just contains a forward declaration:

square.h:

```
#ifndef SQUARE_H
#define SQUARE_H

int getSquareSides(); // forward declaration for getSquareSides
int getSquarePerimeter(int sideLength); // forward declaration for
getSquarePerimeter

#endif
```

square.cpp:

```
#include "square.h"

int getSquareSides() // actual definition for getSquareSides
{
    return 4;
```

```
}

int getSquarePerimeter(int sideLength)
{
    return sideLength * getSquareSides();
}
```

main.cpp:

```
#include "square.h" // square.h is also included once here
#include <iostream>

int main()
{
    std::cout << "a square has " << getSquareSides() << " sides\n";
    std::cout << "a square of length 5 has perimeter length " <<
getSquarePerimeter(5) << '\n';

    return 0;
}
```

Now when the program is compiled, function *getSquareSides* will have just one definition (via *square.cpp*), so the linker is happy. File *main.cpp* is able to call this function (even though it lives in *square.cpp*) because it includes *square.h*, which has a forward declaration for the function (the linker will connect the call to *getSquareSides* from *main.cpp* to the definition of *getSquareSides* in *square.cpp*).

Can't we just avoid definitions in header files?

We've generally told you not to include function definitions in your headers. So you may be wondering why you should include header guards if they protect you from something you shouldn't do.

There are quite a few cases we'll show you in the future where it's necessary to put non-function definitions in a header file. For example, C++ will let you create your own types. These custom types are typically defined in header files, so the type definitions can be propagated out to the code files that need to use them. Without a header guard, a code file could end up with multiple (identical) copies of a given type definition, which the compiler will flag as an error.

So even though it's not strictly necessary to have header guards at this point in the tutorial series, we're establishing good habits now, so you don't have to unlearn bad habits later.

#pragma once

Modern compilers support a simpler, alternate form of header guards using the `#pragma` preprocessor directive:

```
#pragma once

// your code here
```

`#pragma once` serves the same purpose as header guards: to avoid a header file from being included multiple times. With traditional header guards, the developer is responsible for guarding the header (by using preprocessor directives `#ifndef`, `#define`, and `#endif`). With `#pragma once`, we're requesting that the compiler guard the header. How exactly it does this is an implementation-specific detail.

> **For advanced readers**
>
> There is one known case where `#pragma once` will typically fail. If a header file is copied so that it exists in multiple places on the file system, if somehow both copies of the header get included, header guards will successfully de-dupe the identical headers, but `#pragma once` won't (because the compiler won't realize they are actually identical content).

For most projects, `#pragma once` works fine, and many developers now prefer it because it is easier and less error-prone. Many IDEs will also auto-include `#pragma once` at the top of a new header file generated through the IDE.

> **Warning**
>
> The `#pragma` directive was designed for compiler implementers to use for whatever purposes they desire. As such, which pragmas are supported and what meaning those pragmas have is completely implementation-specific. With the exception of `#pragma once`, do not expect a pragma that works on one compiler to be supported by another.
>
> Because `#pragma once` is not defined by the C++ standard, it is possible that some compilers may not implement it. For this reason, some development houses (such as Google) recommend using traditional header guards. In this tutorial series, we will favor header guards, as they are the most conventional way to guard headers. However, support for `#pragma once` is fairly ubiquitous at this point, and if you wish to use `#pragma once` instead, that is generally accepted in modern C++.

Summary

Header guards are designed to ensure that the contents of a given header file are not copied more than once into any single file, in order to prevent duplicate definitions.

Duplicate *declarations* are fine – but even if your header file is composed of all declarations (no definitions) it's still a best practice to include header guards.

Note that header guards do *not* prevent the contents of a header file from being copied (once) into separate project files. This is a good thing, because we often need to reference the contents of a given header from different project files.

02.13 — How to design your first programs

Now that you've learned some basics about programs, let's look more closely at *how* to design a program.

When you sit down to write a program, generally you have some kind of idea, which you'd like to write a program for. New programmers often have trouble figuring out how to convert that idea into actual code. But it turns out, you have many of the problem solving skills you need already, acquired from everyday life.

The most important thing to remember (and hardest thing to do) is to design your program *before you start coding*. In many regards, programming is like architecture. What would happen if you tried to build a house without following an architectural plan? Odds are, unless you were very talented, you'd end up with a house that had a lot of problems: walls that weren't straight, a leaky roof, etc… Similarly, if you try to program before you have a good game-plan moving forward, you'll likely find that your code has a lot of problems, and you'll have to spend a lot of time fixing problems that could have been avoided altogether with a little thinking ahead.

A little up-front planning will save you both time and frustration in the long run.

In this lesson, we'll lay out a generalized approach for converting ideas into simple functional programs.

Design step 1: Define your goal

In order to write a successful program, you first need to define what your goal is. Ideally, you should be able to state this in a sentence or two. It is often useful to express this as a user-facing outcome. For example:

- Allow the user to organize a list of names and associated phone numbers.
- Generate randomized dungeons that will produce interesting looking caverns.
- Generate a list of stock recommendations for stocks that have high dividends.
- Model how long it takes for a ball dropped off a tower to hit the ground.

Although this step seems obvious, it's also highly important. The worst thing you can do is write a program that doesn't actually do what you (or your boss) wanted!

Design step 2: Define requirements

While defining your problem helps you determine *what* outcome you want, it's still vague. The next step is to think about requirements.

Requirements is a fancy word for both the constraints that your solution needs to abide by (e.g. budget, timeline, space, memory, etc…), as well as the capabilities that the program must exhibit in order to meet the users' needs. Note that your requirements should similarly be focused on the "what", not the "how".

For example:

- Phone numbers should be saved, so they can be recalled later.
- The randomized dungeon should always contain a way to get from the entrance to an exit.
- The stock recommendations should leverage historical pricing data.
- The user should be able to enter the height of the tower.
- We need a testable version within 7 days.
- The program should produce results within 10 seconds of the user submitting their request.
- The program should crash in less than 0.1% of user sessions.

A single problem may yield many requirements, and the solution isn't "done" until it satisfies all of them.

Design step 3: Define your tools, targets, and backup plan

When you are an experienced programmer, there are many other steps that typically would take place at this point, including:

- Defining what target architecture and/or OS your program will run on.
- Determining what set of tools you will be using.
- Determining whether you will write your program alone or as part of a team.
- Defining your testing/feedback/release strategy.
- Determining how you will back up your code.

However, as a new programmer, the answers to these questions are typically simple: You are writing a program for your own use, alone, on your own system, using an IDE you downloaded, and your code is probably not used by anybody but you. This makes things easy.

That said, if you are going to work on anything of non-trivial complexity, you should have a plan to backup your code. It's not enough to zip or copy your source directory to another location on the same storage device – if your storage device dies or becomes corrupted, you'll lose everything. Copying or zipping to removable storage (e.g. a flash drive) is better, though you still risk losing everything in the event of theft, fire, or a significant natural disaster.

The best backup strategy involves getting a copy of your code onto a machine that exists in a different physical location. There are lots of easy ways to do this: Zip it up and email it to yourself, upload it to a cloud storage service (e.g. Dropbox), use a file transfer protocol (e.g. SFTP) to upload it to a server you control, or use a version control system residing on another machine or in the cloud (e.g. github). Version control systems have the added advantage of not only being able to restore your files, but also to roll them back to a previous version.

Design step 4: Break hard problems down into easy problems

In real life, we often need to perform tasks that are very complex. Trying to figure out how to do these tasks can be very challenging. In such cases, we often make use of the **top down** method of problem

solving. That is, instead of solving a single complex task, we break that task into multiple subtasks, each of which is individually easier to solve. If those subtasks are still too difficult to solve, they can be broken down further. By continuously splitting complex tasks into simpler ones, you can eventually get to a point where each individual task is manageable, if not trivial.

Let's take a look at an example of this. Let's say we want to clean our house. Our task hierarchy currently looks like this:

- Clean the house

Cleaning the entire house is a pretty big task to do in one sitting, so let's break it into subtasks:

- Clean the house
  - Vacuum the carpets
  - Clean the bathrooms
  - Clean the kitchen

That's more manageable, as we now have subtasks that we can focus on individually. However, we can break some of these down even further:

- Clean the house
  - Vacuum the carpets
  - Clean the bathrooms
    - Scrub the toilet (yuck!)
    - Wash the sink
  - Clean the kitchen
    - Clear the countertops
    - Clean the countertops
    - Scrub the sink
    - Take out the trash

Now we have a hierarchy of tasks, none of them particularly hard. By completing each of these relatively manageable sub-items, we can complete the more difficult overall task of cleaning the house.

The other way to create a hierarchy of tasks is to do so from the **bottom up**. In this method, we'll start from a list of easy tasks, and construct the hierarchy by grouping them.

As an example, many people have to go to work or school on weekdays, so let's say we want to solve the problem of "go to work". If you were asked what tasks you did in the morning to get from bed to work, you might come up with the following list:

- Pick out clothes
- Get dressed
- Eat breakfast
- Travel to work
- Brush your teeth
- Get out of bed
- Prepare breakfast
- Get on your bicycle
- Take a shower

Using the bottom up method, we can organize these into a hierarchy of items by looking for ways to group items with similarities together:

- Get from bed to work

    - Bedroom things
        - Turn off alarm
        - Get out of bed
        - Pick out clothes
    - Bathroom things
        - Take a shower
        - Get dressed
        - Brush your teeth
    - Breakfast things
        - Make coffee or tea
        - Eat cereal
    - Transportation things
        - Get on your bicycle
        - Travel to work

As it turns out, these task hierarchies are extremely useful in programming, because once you have a task hierarchy, you have essentially defined the structure of your overall program. The top level task (in this case, “Clean the house” or “Go to work”) becomes `main()` (because it is the main problem you are trying to solve). The subitems become functions in the program.

If it turns out that one of the items (functions) is too difficult to implement, simply split that item into multiple sub-items/sub-functions. Eventually you should reach a point where each function in your program is trivial to implement.

Design step 5: Figure out the sequence of events

Now that your program has a structure, it’s time to determine how to link all the tasks together. The first step is to determine the sequence of events that will be performed. For example, when you get up in the morning, what order do you do the above tasks? It might look like this:

- Bedroom things
- Bathroom things
- Breakfast things
- Transportation things

If we were writing a calculator, we might do things in this order:

- Get first number from user
- Get mathematical operation from user
- Get second number from user
- Calculate result
- Print result

At this point, we’re ready for implementation.

Implementation step 1: Outlining your main function

Now we’re ready to start implementation. The above sequences can be used to outline your main program. Don’t worry about inputs and outputs for the time being.

```
int main()
{
//    doBedroomThings();
//    doBathroomThings();
//    doBreakfastThings();
//    doTransportationThings();

    return 0;
}
```

Or in the case of the calculator:

```
int main()
{
    // Get first number from user
//    getUserInput();

    // Get mathematical operation from user
//    getMathematicalOperation();

    // Get second number from user
//    getUserInput();

    // Calculate result
//    calculateResult();

    // Print result
//    printResult();

    return 0;
}
```

Note that if you're going to use this "outline" method for constructing your programs, your functions won't compile because the definitions don't exist yet. Commenting out the function calls until you're ready to implement the function definitions is one way to address this (and the way we'll show here). Alternatively, you can *stub out* your functions (create placeholder functions with empty bodies) so your program will compile.

Implementation step 2: Implement each function

In this step, for each function, you'll do three things:

1. Define the function prototype (inputs and outputs)
2. Write the function
3. Test the function

If your functions are granular enough, each function should be fairly simple and straightforward. If a given function still seems overly-complex to implement, perhaps it needs to be broken down into subfunctions that can be more easily implemented (or it's possible you did something in the wrong order, and need to revisit your sequencing of events).

Let's do the first function from the calculator example:

```
#include <iostream>
```

```
// Full implementation of the getUserInput function
int getUserInput()
{
    std::cout << "Enter an integer: ";
    int input{};
    std::cin >> input;

    return input;
}

int main()
{
    // Get first number from user
    int value{ getUserInput() }; // Note we've included code here to
test the return value!
    std::cout << value << '\n'; // debug code to ensure
getUserInput() is working, we'll remove this later

    // Get mathematical operation from user
//    getMathematicalOperation();

    // Get second number from user
//    getUserInput();

    // Calculate result
//    calculateResult();

    // Print result
//    printResult();

    return 0;
}
```

First, we've determined that the *getUserInput* function takes no arguments, and will return an int value back to the caller. That gets reflected in the function prototype having a return value of int and no parameters. Next, we've written the body of the function, which is a straightforward 4 statements. Finally, we've implemented some temporary code in function *main* to test that function *getUserInput* (including its return value) is working correctly.

We can run this program many times with different input values and make sure that the program is behaving as we expect at this point. If we find something that doesn't work, we know the problem is in the code we've just written.

Once we're convinced the program is working as intended up to this point, we can remove the temporary testing code, and proceed to implementation of the next function (function *getMathematicalOperation*). We won't finish the program in this lesson, as we need to cover some additional topics first.

Remember: Don't implement your entire program in one go. Work on it in steps, testing each step along the way before proceeding.

We cover testing in more detail in lesson 9.1 – Introduction to testing your code.

Implementation step 3: Final testing

Once your program is “finished”, the last step is to test the whole program and ensure it works as intended. If it doesn’t work, fix it.

Words of advice when writing programs

Keep your programs simple to start. Often new programmers have a grand vision for all the things they want their program to do. “I want to write a role-playing game with graphics and sound and random monsters and dungeons, with a town you can visit to sell the items that you find in the dungeon”. If you try to write something too complex to start, you will become overwhelmed and discouraged at your lack of progress. Instead, make your first goal as simple as possible, something that is definitely within your reach. For example, “I want to be able to display a 2-dimensional field on the screen”.

Add features over time. Once you have your simple program working and working well, then you can add features to it. For example, once you can display your field, add a character who can walk around. Once you can walk around, add walls that can impede your progress. Once you have walls, build a simple town out of them. Once you have a town, add merchants. By adding each feature incrementally your program will get progressively more complex without overwhelming you in the process.

Focus on one area at a time. Don’t try to code everything at once, and don’t divide your attention across multiple tasks. Focus on one task at a time. It is much better to have one working task and five that haven’t been started yet than six partially-working tasks. If you split your attention, you are more likely to make mistakes and forget important details.

Test each piece of code as you go. New programmers will often write the entire program in one pass. Then when they compile it for the first time, the compiler reports hundreds of errors. This can not only be intimidating, if your code doesn’t work, it may be hard to figure out why. Instead, write a piece of code, and then compile and test it immediately. If it doesn’t work, you’ll know exactly where the problem is, and it will be easy to fix. Once you are sure that the code works, move to the next piece and repeat. It may take longer to finish writing your code, but when you are done the whole thing should work, and you won’t have to spend twice as long trying to figure out why it doesn’t.

Don’t invest in perfecting early code. The first draft of a feature (or program) is rarely good. Furthermore, programs tend to evolve over time, as you add capabilities and find better ways to structure things. If you invest too early in polishing your code (adding lots of documentation, full compliance with best practices, making optimizations), you risk losing all of that investment when a code change is necessary. Instead, get your features minimally working and then move on. As you gain confidence in your solutions, apply successive layers of polish. Don’t aim for perfect – non-trivial programs are never perfect, and there’s always something more that could be done to improve them. Get to “good enough” and move on.

Optimize for maintainability, not performance. There is a famous quote (by Donald Knuth) that says “premature optimization is the root of all evil”. New programmers often spend far too much time thinking about how to micro-optimize their code (e.g. trying to figure out which

of 2 statements is faster). This rarely matters. Most performance benefits come from good program structure, using the right tools and capabilities for the problem at hand, and following best practices. Additional time should be used to improve the maintainability of your code. Find redundancy and remove it. Split up long functions into shorter ones. Replace awkward or hard to use code with something better. The end result will be code that is easier to improve and optimize later (after you've determined where optimization is actually needed) and fewer bugs. We offer some additional suggestions in lesson 3.10 – Finding issues before they become problems.

> A complex system that works is invariably found to have evolved from a simple system that worked
>
> —John Gall, Systemantics: How Systems Really Work and How They Fail p. 71

Conclusion

Many new programmers shortcut the design process (because it seems like a lot of work and/or it's not as much fun as writing the code). However, for any non-trivial project, following these steps will save you a lot of time in the long run. A little planning up front saves a lot of debugging at the end.

> **Key insight**
>
> Spending a little time up front thinking about how to structure your program will lead to better code and less time spent finding and fixing errors.

> I would say this is arguably the most important thing in programming and some of us, like me at first, took it for granted.
>
> —Reader Emeka Daniel, comment on learncpp.com

As you become more comfortable with these concepts and tips, they will start coming more naturally to you. Eventually you will get to the point where you can write entire functions (and short programs) with minimal pre-planning.

02.x — Chapter 2 summary and quiz

Chapter Review

A **function** is a reusable sequence of statements designed to do a particular job. Functions you write yourself are called **user-defined** functions.

A **function call** is an expression that tells the CPU to execute a function. The function initiating the function call is the **caller**, and the function being called is the **callee** or **called** function. Do not forget to include parenthesis when making a function call.

The curly braces and statements in a function definition are called the **function body**.

A function that returns a value is called a **value-returning function**. The **return type** of a function indicates the type of value that the function will return. The **return statement** determines the specific **return value** that is returned to the caller. A return value is copied from the function back to the caller – this process is called **return by value**. Failure to return a value from a non-void function will result in undefined behavior.

The return value from function *main* is called a **status code**, and it tells the operating system (and any other programs that called yours) whether your program executed successfully or not. By consensus a return value of 0 means success, and a non-zero return value means failure.

Practice **DRY** programming – "don't repeat yourself". Make use of variables and functions to remove redundant code.

Functions with a return type of **void** do not return a value to the caller. A function that does not return a value is called a **void function** or **non-value returning function**. Void functions can't be called where a value is required.

A return statement that is not the last statement in a function is called an **early return**. Such a statement causes the function to return to the caller immediately.

A **function parameter** is a variable used in a function where the value is provided by the caller of the function. An **argument** is the specific value passed from the caller to the function. When an argument is copied into the parameter, this is called **pass by value**.

Function parameters and variables defined inside the function body are called **local variables**. The time in which a variable exists is called its **lifetime**. Variables are created and destroyed at **runtime**, which is when the program is running. A variable's **scope** determines where it can be seen and used. When a variable can be seen and used, we say it is **in scope**. When it can not be seen, it can not be used, and we say it is **out of scope**. Scope is a **compile-time** property, meaning it is enforced at compile time.

Whitespace refers to characters used for formatting purposes. In C++, this includes spaces, tabs, and newlines.

A **forward declaration** allows us to tell the compiler about the existence of an identifier before actually defining the identifier. To write a forward declaration for a function, we use a **function prototype**, which includes the function's return type, name, and parameters, but no function body, followed by a semicolon.

A **definition** actually implements (for functions and types) or instantiates (for variables) an identifier. A **declaration** is a statement that tells the compiler about the existence of the identifier. In C++, all definitions serve as declarations. **Pure declarations** are declarations that are not also definitions (such as function prototypes).

Most non-trivial programs contain multiple files.

When two identifiers are introduced into the same program in a way that the compiler or linker can't tell them apart, the compiler or linker will error due to a **naming collision**. A **namespace** guarantees that all identifiers within the namespace are unique. The std namespace is one such namespace.

The **preprocessor** is a process that runs on the code before it is compiled. **Directives** are special instructions to the preprocessor. Directives start with a # symbol and end with a newline. A **macro** is a rule that defines how input text is converted to a replacement output text.

Header files are files designed to propagate declarations to code files. When using the *#include* directive, the *#include* directive is replaced by the contents of the included file. When including headers, use angled brackets when including system headers (e.g. those in the C++ standard library), and use double quotes when including user-defined headers (the ones you write). When including system headers, include the versions with no .h extension if they exist.

Header guards prevent the contents of a header from being included more than once into a given code file. They do not prevent the contents of a header from being included into multiple different code files.

Quiz time

Be sure to use your editor's auto-formatting feature to keep your formatting consistent and make your code easier to read.

Question #1

Write a single-file program (named main.cpp) that reads two separate integers from the user, adds them together, and then outputs the answer. The program should use three functions:

- A function named "readNumber" should be used to get (and return) a single integer from the user.
- A function named "writeAnswer" should be used to output the answer. This function should take a single parameter and have no return value.
- A main() function should be used to glue the above functions together.

> **Hint:** You do not need to write a separate function to do the adding (just use operator+ directly).

> **Hint:** You will need to call readNumber() twice.

main.cpp:

```
#include <iostream>

int readNumber()
{
    std::cout << "Enter a number to add: ";
    int x {};
    std::cin >> x;
    return x;
```

```
}

void writeAnswer(int x)
{
    std::cout << "The answer is " << x << '\n';
}

int main()
{
    int x { readNumber() };
    int y { readNumber() };
    writeAnswer(x + y); // using operator+ to pass the sum of x and
y to writeAnswer()
    return 0;
}
```

Question #2

Modify the program you wrote in exercise #1 so that readNumber() and writeAnswer() live in a separate file called “io.cpp”. Use a forward declaration to access them from main().

If you’re having problems, make sure “io.cpp” is properly added to your project so it gets compiled.

io.cpp:

```
#include <iostream>

int readNumber()
{
    std::cout << "Enter a number to add: ";
    int x {};
    std::cin >> x;
    return x;
}

void writeAnswer(int x)
{
    std::cout << "The answer is " << x << '\n';
}
```

main.cpp:

```
// We don't need to #include <iostream> since main.cpp doesn't use
any input/output functionality

// These are the forward declarations for the functions in io.cpp
int readNumber();
void writeAnswer(int x);

int main()
{
    int x { readNumber() };
    int y { readNumber() };
    writeAnswer(x+y);
    return 0;
}
```

Question #3

Modify the program you wrote in #2 so that it uses a header file (named io.h) to access the functions instead of using forward declarations directly in your code (.cpp) files. Make sure your header file uses header guards.

io.h:

```
#ifndef IO_H
#define IO_H

int readNumber();
void writeAnswer(int x);

#endif
```

io.cpp:

```
#include "io.h"
#include <iostream>

int readNumber()
{
    std::cout << "Enter a number to add: ";
    int x {};
    std::cin >> x;
    return x;
}

void writeAnswer(int x)
{
    std::cout << "The answer is " << x << '\n';
}
```

main.cpp:

```
#include "io.h"

int main()
{
    int x { readNumber() };
    int y { readNumber() };
    writeAnswer(x+y);
    return 0;
}
```

While technically *io.cpp* doesn't require the inclusion of *io.h*, it is a best practice for code files to include their paired header files. This is covered in lesson 2.11 – Header files.

If you compile your program and get errors like one of these:

```
unresolved external symbol "int __cdecl readNumber(void)" (?
readNumber@@YAHXZ)
undefined reference to `readNumber()'
```

Then you probably forgot to include `io.cpp` in your project, so the definitions for `readNumber()` (and `writeAnswer()`) aren't being compiled into your project.